



**Spécialité : DUT en Sécurité Informatique et
Réseaux (SIR)**

Rapport de Projet de Fin d'Etude (PFE)

Sous le thème

**Détection des cybermenaces dans un Cloud IaaS virtualisé:
Déploiement et intégration de Wazuh SIEM**

Effectué par :

**Salma Fadili
Imane Rajif
Hajar Rachid**

Encadré par :

M. Hanaa Attou

Présenté et soutenu le 19/06/2026, devant le Jury composé de :

Pr. N. HAFIDI

Pr. M. HANAA ATTOU

Président

Encadrante

Année Universitaire : 2025-2026

Remerciements

Avant tout, nous remercions Dieu le Tout-Puissant De nous avoir accordé la santé, la volonté et le courage pour accomplir ce travail.

Nous tenons à exprimer notre profonde gratitude à notre encadrante, Madame **Hanaa Attou**, pour son accompagnement bienveillant, sa disponibilité, ses orientations pertinentes, ses précieux conseils tout au long de la réalisation de ce projet de fin d'études.

Nous remercions également l'ensemble du corps **professoral** et **administratif** de notre établissement pour la qualité de la formation dispensée durant notre cursus.

Nos sincères remerciements vont aussi aux **membres du jury** qui nous font l'honneur d'évaluer ce travail.

Enfin, nous remercions toutes **les personnes** qui, de près ou de loin, ont contribué à l'aboutissement de ce projet.

Résumé

Ce projet porte sur la détection et la sécurisation d'un environnement Cloud privé IaaS virtualisé face aux cybermenaces. Nous avons conçu une infrastructure distribuée composée de trois machines interconnectées via ZeroTier One : une machine attaquante sous Kali Linux, une machine cible sous Ubuntu Server hébergeant Apache, MySQL et PFSense, et un serveur de supervision déployé via Docker.

Trois scénarios d'attaques ont été simulés : reconnaissance via Nmap, scan de vulnérabilités via Nikto, et force brute SSH via Hydra. Ces attaques ont été détectées en temps réel par Wazuh SIEM, avec envoi de notifications instantanées via un Bot Telegram grâce à un script Python personnalisé.

Mots-clés : Cloud IaaS, Cybersécurité, Wazuh SIEM, ZeroTier, Docker, Nmap, Nikto, Hydra, Python, Telegram.

Abstract

This project focuses on the detection and security of a virtualized private IaaS Cloud environment against cyber threats. We designed a distributed infrastructure composed of three machines interconnected via ZeroTier One : an attacking machine running Kali Linux, a target machine running Ubuntu Server hosting Apache, MySQL and PFSense, and a supervision server deployed via Docker.

Three attack scenarios were simulated : reconnaissance via Nmap, vulnerability scanning via Nikto, and SSH brute force via Hydra. These attacks were detected in real time by Wazuh SIEM, with instant notifications sent via a Telegram Bot through a custom Python script.

Keywords : Cloud IaaS, Cybersecurity, Wazuh SIEM, ZeroTier, Docker, Nmap, Nikto, Hydra, Python, Telegram.

Sommaire

Remerciements	1
Résumé	2
Abstract	3
Listes des figures.....	7
Listes des Tableaux	9
Listes des abréviations	10
Introduction Générale.....	11
Chapitre 1 : Cloud Computing et sécurité des environnements Cloud	12
1.1 Présentation du Cloud Computing.....	13
1.1.1 Définition du Cloud	13
1.1.2 Caractéristique principales du cloud.....	13
1.1.3 Évolution du Cloud Computing :	14
1.2 Les modules de Cloud Computing	14
1.2.1 IaaS Infrastructure en tant que service	15
1.2.2 PaaS Plate-forme en tant que service	15
1.2.3 SaaS Application en tant que Service	16
1.3 Les formes de déploiement du Cloud Computing.....	18
1.3.1 Le Cloud public	18
1.3.2 le cloude privé.....	18
1.3.3 le cloud hybride	18
1.4 La virtualisation	18
1.4.1 Définition et principes	19
1.4.2 Types d'hyperviseurs	19
1.4.3 Rôle de la virtualisation dans le Cloud	20
1.5 Sécurité dans le Cloud Computing.....	21
1.5.1 Enjeux de la sécurité	21
1.5.2 les services de la sécurité (CIA).....	22
1.5.3 Les mécanismes de sécurité	23
1.6 Les cybermenaces ciblant le cloud.....	25
1.6.1 Menaces et vulnérabilités	25
1.6.2 le déni de service (Dos/DdoS).....	26
1.6.3 L'usurpation d'adresse (IP Spoofing)	27

1.6.4 Attaque d'un site web (SQLI ,XSS).....	27
1.6.5 Méthodologie d'une attaque	28
1.7 Méthodes de défense	28
Chapitre 2 : Conception de l'architecture et présentation des outils	30
2.1 Contexte et Justification du Projet.....	31
2.1.1 Motivation du projet.....	31
2.1.2 Problématique	31
2.2 Analyse des besoins et spécifications	32
2.2.1 Besoins fonctionnels	32
2.2.2 Besoins non-fonctionnels	32
2.3 Architecture cible proposée	33
2.4 Étude comparative et choix des outils	34
2.4.1 Environnement de virtualisation et réseau	35
2.4.2 Plateforme offensive (Red Team).....	35
2.4.3 Plateforme cible et services applicatifs	36
2.4.4 Solution de supervision et d'alerte (Blue Team).....	37
Chapitre 3 : Mise en œuvre et Tests	39
3.1 Mise en place du réseau virtuel.....	40
3.1.1 Présentation de ZeroTier One	40
3.1.2 Création et configuration du réseau ZeroTier	40
3.1.3 Vérification de la connectivité.....	41
3.2 Configuration des machines.....	43
3.2.1 Configuration de la machine Ubuntu Server (Cible).....	43
3.2.2. Configuration et Filtrage via le Pare-feu pfSense.....	44
3.2.3. Installation et Configuration de l'Agent Wazuh.....	46
3.2.4 Configuration de Kali Linux (Machine attaquante)	47
3.3 Déploiement de Wazuh SIEM.....	49
3.3.1 Présentation de Wazuh	49
3.3.2 Installation via Docker Compose	49
3.3.3 Connexion de l'Agent au Manager.....	50
3.3.4 Règles de détection utilisées.....	50
3.4 Système de notification : Script Python « custom-telegram	51
3.4.1 Objectif et motivation	51
3.4.2 Création du Bot Telegram.....	51
3.4.3 Code Python « custom-telegram ».....	52

3.4.4 Configuration dans ossec.conf	52
3.4.5 Résultat : Notification reçue sur Telegram	53
3.5 Méthodologie des tests.....	54
3.6 Scénario 1: Reconnaissance (Nmap).....	54
3.6.1 Présentation de Nmap	54
3.6.2 Commande utilisée.....	55
3.6.3 Détection par Wazuh	55
3.6.4 Analyse.....	57
3.7 Scénario 2 : Brute Force SSH (Hydra)	57
3.7.1 Présentation de Hydra	57
3.7.2 Préparation	57
3.7.3 Commande utilisée.....	57
3.7.4 Détection par Wazuh	58
3.7.5 Notification Telegram	60
3.7.6 Analyse.....	60
3.8 Scénario 3 : Scan de vulnérabilités (Nikto)	60
3.8.1 Présentation de Nikto.....	60
3.8.2 Commande utilisée.....	60
3.8.3 Détection par Wazuh	61
3.8.4 Notification Telegram	62
3.9 Tableau comparatif des résultats	63
3.10 Évaluation de l'efficacité du système	63
Conclusion générale	65
Bibliographie.....	66

Listes des figures

Figure 1: Les couches de Cloud Computing	14
Figure 2: Répartition des responsabilités [5]	17
Figure 3: Hyperviseur type 1	19
Figure 4: Hyperviseur type 2	20
Figure 5: Les services de la sécurité (CIA)	23
Figure 6: Architecture du projet.....	34
Figure 7: Logo VirtualBox	35
Figure 8: Logo ZeroTier One	35
Figure 9: Logo Kali Linux.....	35
Figure 10: Logo Nmap	36
Figure 11: Logo Nikto	36
Figure 12:: Logo Hydra.....	36
Figure 13: Logo Ubuntu Server	36
Figure 14: Logo Apache & MySQL	37
Figure 15: Logo PFSense.....	37
Figure 16: Logo Wazuh.....	37
Figure 17: Logo Docker.....	38
Figure 18: Logo Telegram	38
Figure 19: Tableau de bord ZeroTier One	41
Figure 20: Test de connectivité Kali → Ubuntu	42
Figure 21: Test de connectivité Ubuntu → Wazuh.....	42
Figure 22: Test de connectivité Wazuh → Ubuntu.....	42
Figure 23: Installation du serveur Apache2.....	43
Figure 24: Installation de MySQL	44
Figure 25: Installation et activation du service SSH sur Ubuntu Server.....	44

Figure 26: Configuration pfSense (règles)	45
Figure 27: État service wazuh-agent	47
Figure 28: Vérification Nmap sur Kali.....	48
Figure 29: Vérification Hydra sur Kali	48
Figure 30: Vérification Nikto sur Kali.....	48
Figure 31: Résultat commande docker ps.....	50
Figure 32: Création du Bot Telegram	52
Figure 33: Script Python custom-telegram	52
Figure 34: Configuration bloc integration ossec.conf	53
Figure 35: Notification Telegram reçue.....	53
Figure 36: Capture scan Nmap depuis Kali.....	55
Figure 37:Dashboard Wazuh : Détection Nmap	56
Figure 38: Alertes Wazuh — scan Nmap.....	56
Figure 39: Capture Hydra en cours d'exécution	58
Figure 40: Dashboard Wazuh — Détection Hydra	59
Figure 41 : :Alertes Wazuh — Brute Force SSH.....	59
Figure 42: Notification Telegram (Brute Force).....	60
Figure 43: Capture scan Nikto depuis Kali.....	61
Figure 44: Dashboard Wazuh (Détection Nikto).....	61
Figure 45: Alertes Wazuh (scan Nikto)	62
Figure 46: Notification Telegram (Nikto)	62

Listes des Tableaux

Tableau 1 : Étapes de configuration de ZeroTier One.....	41
Tableau 2 : Composants de la solution Wazuh	49
Tableau 3 : Règles Wazuh utilisées pour la détection SSH	51
Tableau 4 : Matrice de synthèse des scénarios d'attaques simulés	54
Tableau 5 : comparatif des résultats	63

Listes des abréviations

- **CC** : Cloud Computing
- **CLI** : Command Line Interface
- **CPU** : Central Processing Unit
- **DDoS** : Distributed Denial of Service
- **DMZ** : Demilitarized Zone
- **DNS** : Domain Name System
- **DoS** : Denial of Service
- **HTTP** : HyperText Transfer Protocol
- **IaaS** : Infrastructure as a Service
- **IDS** : Intrusion Detection System
- **IPS** : Intrusion Prevention System
- **IP** : Internet Protocol
- **OS** : Operating System
- **PaaS** : Platform as a Service
- **PFE** : Projet de Fin d'Études
- **SaaS** : Software as a Service
- **SDN** : Software Defined Network
- **SIEM** : Security Information and Event Management
- **SSH** : Secure Shell
- **SSL** : Secure Sockets Layer
- **VM** : Virtual Machine
- **VPN** : Virtual Private Network

Introduction Générale

À l'ère du numérique, le Cloud Computing (CC) s'est imposé comme le socle des infrastructures informatiques modernes, offrant aux entreprises et aux organisations une flexibilité, une scalabilité et une accessibilité sans précédent. Parmi ses modèles de service, la couche IaaS (Infrastructure as a Service) occupe une place centrale, car elle permet une gestion directe des machines virtuelles (VM), des réseaux et des systèmes d'exploitation, constituant ainsi la base sur laquelle repose l'intégralité de notre projet.

Cependant, cette évolution s'accompagne de risques croissants. Les environnements Cloud virtualisés sont aujourd'hui exposés à des cybermenaces de plus en plus sophistiquées, notamment les attaques par force brute, les scans de vulnérabilités et les dénis de service, qui menacent directement la confidentialité, l'intégrité et la disponibilité des données hébergées.

C'est dans ce contexte que s'inscrit notre projet intitulé **"Détection des cybermenaces dans un Cloud IaaS virtualisé : Déploiement et intégration de Wazuh SIEM"**. Notre objectif est de concevoir un environnement Cloud privé IaaS simulé, composé de trois machines interconnectées via ZeroTier One, et d'y intégrer la solution SIEM Wazuh pour la détection en temps réel des intrusions, couplée à un script Python personnalisé permettant une notification instantanée via Telegram.

Le projet s'est déroulé en trois phases successives : une phase de mise en place de l'infrastructure, une phase de simulation d'attaques réelles (Nmap, Hydra, Nikto), et une phase de détection, d'analyse et d'évaluation des résultats obtenus.

Ce rapport s'articule autour de trois chapitres :

- Le premier présente les concepts fondamentaux du CC et les enjeux de sécurité des environnements virtualisés.
- Le deuxième expose la conception de l'architecture et les outils technologiques retenus.
- Le troisième décrit la mise en œuvre pratique, les tests d'intrusion simulés et les résultats obtenus.

Chapitre 1 : Cloud Computing et sécurité des environnements Cloud

Le CC est devenu l'épine dorsale des infrastructures modernes, offrant une flexibilité et une scalabilité sans précédent. Cependant, cette centralisation des ressources et l'usage intensif de la virtualisation ouvrent la porte à de nouvelles surfaces d'attaque. La sécurité des environnements Cloud ne se limite plus à la simple protection périmétrique, mais exige une vigilance constante face à des cybermenaces de plus en plus sophistiquées, telles que les scans de vulnérabilités, les accès non autorisés et les forces brutes.

C'est précisément au cœur de cette **problématique** que s'inscrit notre travail. Comment peut-on assurer la visibilité et la détection en temps réel de ces menaces au sein d'une infrastructure virtualisée pour garantir les piliers de la sécurité (CIA) ? Pour répondre à cela, ce chapitre posera d'abord les bases théoriques du Cloud et de la virtualisation, avant d'analyser les mécanismes de défense et les cybermenaces qui ciblent ces architectures.

1.1 Présentation du Cloud Computing

1.1.1 Définition du Cloud

Le CC est la disponibilité à la demande de ressources informatiques (telles que le stockage et l'infrastructure) en tant que services sur Internet. Ainsi, les particuliers et les entreprises n'ont plus besoin de gérer eux-mêmes leurs ressources physiques, et de ne payer que ce qu'ils utilisent. [1]

1.1.2 Caractéristique principales du cloud

Les systèmes de CC présentent plusieurs caractéristiques générales :

- Les ressources informatiques sont mutualisées, ce qui signifie que les utilisateurs (particuliers ou organisations) exploitent un ensemble de ressources généralement hébergées dans des centres de données distants.
- Les ressources sont dynamiques, c'est-à-dire qu'elles peuvent être ajustées (augmentées ou diminuées) selon les besoins, et sont accessibles via Internet ou des réseaux privés.
- L'utilisation des ressources est mesurée, et les utilisateurs sont facturés en fonction de leur consommation réelle.
- Les ressources sont disponibles à la demande et en libre-service, permettant aux utilisateurs de provisionner et de gérer eux-mêmes les ressources informatiques selon leurs besoins.

1.1.3 Évolution du Cloud Computing :

Le CC a émergé avec le développement d'Internet dans les années 1990. Son évolution a été marquée par l'apparition de nouvelles technologies et services numériques, notamment avec des entreprises comme Amazon et Google, qui ont fortement contribué à sa popularisation.

Aujourd'hui, le Cloud connaît une croissance rapide et devient une solution largement adoptée par les entreprises grâce à ses avantages en termes de flexibilité, de performance et de réduction des coûts.[2]

1.2 Les modules de Cloud Computing

La figure ci-dessous représente les trois couches du CC de la couche la moins visible pour les utilisateurs finaux à la plus visible. L'infrastructure as a Service (IaaS) est plutôt gérée par les architectes réseaux, la couche PaaS est destinée aux développeurs d'applications et finalement le logiciel comme un service (SaaS) est le « produit final » pour les utilisateurs.

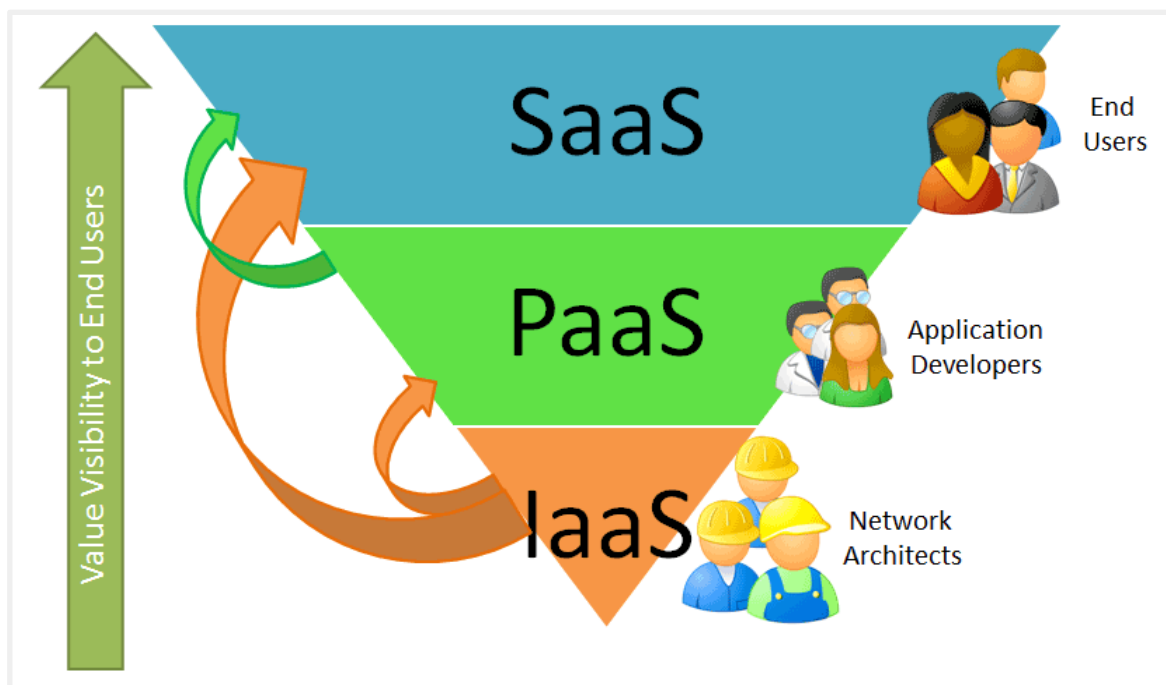


Figure 1: Les couches de Cloud Computing

Cette figure illustre le modèle pyramidal des services du Cloud Computing (SaaS, PaaS, et IaaS) ainsi que les acteurs associés à chaque niveau. À la base, la couche **IaaS (Infrastructure as a Service)** est gérée par les architectes réseau pour fournir les ressources

matérielles et virtuelles brutes. Au milieu, le **PaaS (Platform as a Service)** offre l'environnement nécessaire aux développeurs d'applications. Enfin, au sommet, le **SaaS (Software as a Service)** représente le produit final destiné aux utilisateurs finaux. Plus on descend dans la pyramide, plus le contrôle technique de l'infrastructure augmente.

1.2.1 IaaS Infrastructure en tant que service

Infrastructure as a Service (IaaS) ¹offre un accès à la demande aux services d'infrastructure informatique, dont le calcul, le stockage et la mise en réseau ainsi que la virtualisation. Ce modèle offre le plus haut niveau de contrôle sur vos ressources informatiques et se rapproche le plus des ressources informatiques sur site traditionnelles.[3]

a. Avantages

- Éviter l'achat et la gestion de matériel.
- Offre une grande flexibilité.
- Contrôle total des systèmes (administration à distance par ssh ou RDP).
- Capacité de stockage infini.

b. Inconvénients

- Besoin d'administrateurs système comme pour les solutions serveurs classiques sur site.
- Demande pour les acteurs du cloud des investissements très élevés

c. Prestataires

AmazonEC2, RackSpace, Terremark, GoGrid.

1.2.2 PaaS Plate-forme en tant que service

Platform as a Service (PaaS) ²fournit toutes les ressources matérielles et logicielles nécessaires au développement d'applications cloud. Grâce aux PaaS, les entreprises peuvent se concentrer entièrement sur le développement d'applications, sans avoir à gérer ni à entretenir l'infrastructure sous-jacente.

¹ <https://kinsta.com/fr/blog/iaas-vs-paas/>

² <https://scalingo.com/fr/blog/saas-paas-iaas-basics>

a. Avantages

- Facilité à gérer les développements des applications
- Le déploiement est automatisé, pas de logiciel supplémentaire à acheter ou à installer.
- Pas d'infrastructure nécessaire
- Pas d'installation

b. Inconvénients

- L'application doit être programmée pour la plate-forme cible.
- La sécurité des données doit être prise en compte et analysée, car les informations sont stockées hors site.
- La récupération des données peut être difficile.
- Un fournisseur de PaaS moins qu'idéal pourrait laisser votre entreprise frustrée, besoin de bons niveaux de vitesse, de fiabilité et de support pour que PaaS en vaille la peine.

c. Prestataires

Google avec Google App Engine, Microsoft avec Windows Azure, Force.com

1.2.3 SaaS Application en tant que Service

Software as a Service (SaaS) ³fournit une pile d'applications en tant que service complète, de l'infrastructure sous-jacente à la maintenance, en passant par les mises à jour du logiciel d'application. Une solution SaaS correspond généralement à une application d'utilisateur final dont la gestion (maintenance, mise à jour, infrastructure) est entièrement assurée par le fournisseur de service cloud.

a. Avantages

- Pas d'installation
- Pas de mise à jour.
- Plus de licence.
- Paiement à l'usage.
- Facilité de faire le test de nouveaux logiciels.

b. Inconvénients

- Dépendance totale du prestataire.

³ <https://www.sylob.com/fr/blog/5-bonnes-raisons-choisir-erp-saas>

- Difficulté de changement de prestataire.
- Le coût du fonctionnement en mode SaaS peut-être nettement supérieur à l'acquisition de l'application.
- Interruptions de service.

c. Prestataires

Google avec Google Drive, Microsoft avec Office 365.

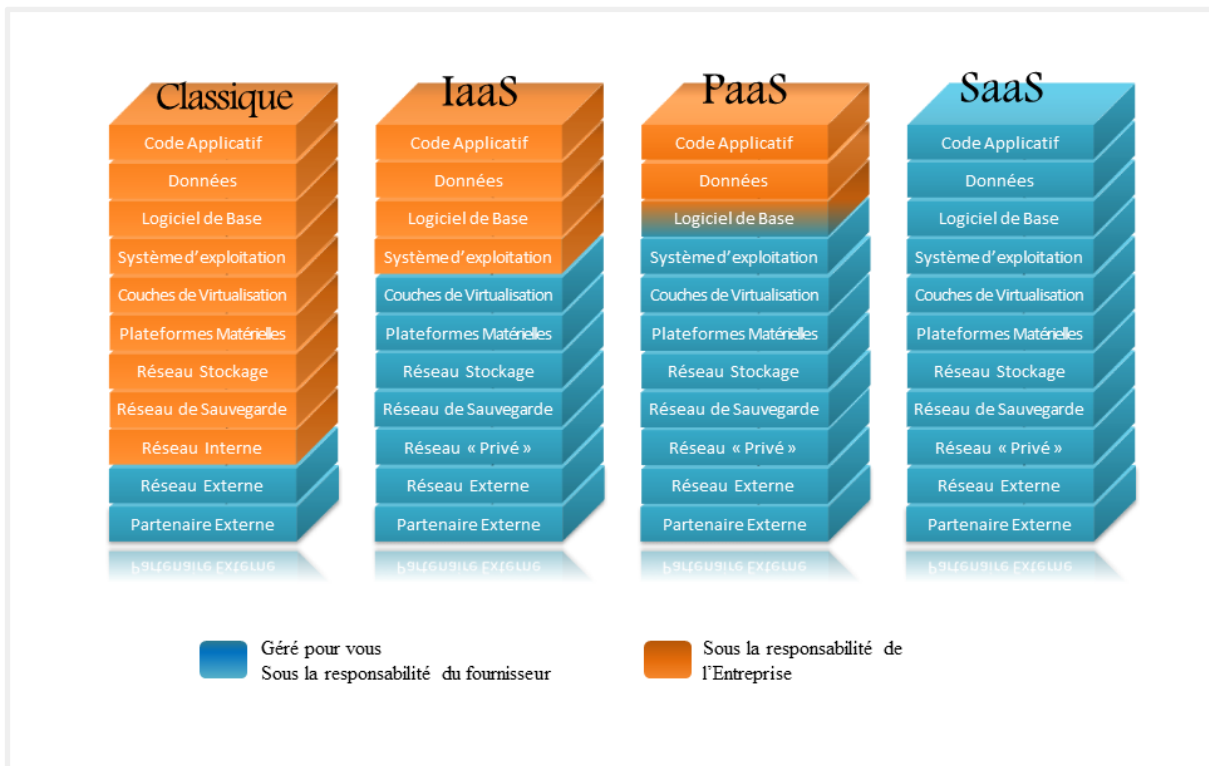


Figure 2: Répartition des responsabilités

Ce schéma met en évidence le concept de responsabilité partagée entre le fournisseur Cloud et l'entreprise (client) selon le modèle de déploiement choisi (Classique, IaaS, PaaS, et SaaS).

- Dans un modèle **Classique (On-Premise)**, l'entreprise gère l'intégralité de la pile informatique, de l'infrastructure physique jusqu'aux applications.
- En mode **IaaS**, le fournisseur prend en charge la partie matérielle et la virtualisation, tandis que l'entreprise reste responsable du système d'exploitation, des données et des applications. C'est ce modèle précis qui est audité et sécurisé dans le cadre de notre projet.

- Plus on évolue vers le **PaaS** puis le **SaaS**, plus la responsabilité de la gestion technique est transférée vers le fournisseur, laissant à l'entreprise uniquement la responsabilité de ses données et de ses configurations applicatives.

Après avoir présenté les modèles de services du CC, il est également important de distinguer les différentes formes de déploiement du Cloud.

1.3 Les formes de déploiement du Cloud Computing

Les entreprises peuvent choisir de déployer des applications sur des Clouds publics, privés ou hybrides. Les intégrateurs Cloud peuvent jouer un rôle essentiel dans la détermination du bon chemin de Cloud pour chaque organisation. Nous distinguons trois formes de CC tels que le Cloud public, également le premier apparu, le Cloud privé et le Cloud hybride qui est en fait la combinaison des deux premiers.[4]

1.3.1 Le Cloud public

Les clouds publics sont exécutés par des fournisseurs de services cloud tiers. Ils proposent des ressources de calcul, de stockage et de réseau sur Internet, ce qui permet aux entreprises d'accéder à des ressources partagées à la demande, en fonction de leurs besoins spécifiques et de leurs objectifs commerciaux

1.3.2 le cloude privé

Les clouds privés sont créés, gérés et détenus par une seule organisation, et hébergés de manière privée dans leurs propres centres de données. Ils sont couramment appelés clouds "sur site". Ils offrent plus de contrôle, de sécurité et de gestion des données, tout en permettant aux utilisateurs internes de bénéficier d'un pool partagé de ressources de calcul, de stockage et de réseau.

1.3.3 le cloud hybride

Les clouds hybrides combinent des modèles de cloud public et privé, ce qui permet aux entreprises d'exploiter des services cloud publics tout en maintenant les fonctionnalités de sécurité et de conformité courantes dans les architectures cloud privées.

1.4 La virtualisation

1.4.1 Définition et principes

La virtualisation est une technologie qui permet de faire fonctionner plusieurs (VM) sur un seul matériel physique. Chaque (VM) peut avoir son propre système (comme Windows, Linux, etc.) et ses programmes, comme s'il s'agissait d'un vrai ordinateur .

1.4.2 Types d'hyperviseurs

En informatique, un hyperviseur est une plate-forme de virtualisation qui permet à plusieurs systèmes d'exploitation de travailler sur une même machine physique en même temps.

Les hyperviseurs sont classés actuellement en deux catégories :

Un **hyperviseur de Type 1**, ou natif, est un logiciel qui s'exécute directement sur une plateforme matérielle, cette plateforme est alors considérée comme outil de contrôle de système d'exploitation. Un système d'exploitation secondaire peut, de ce fait, être exécuté au-dessus du matériel. Linux complet en hyperviseur, sont également considérées comme hyperviseurs de type

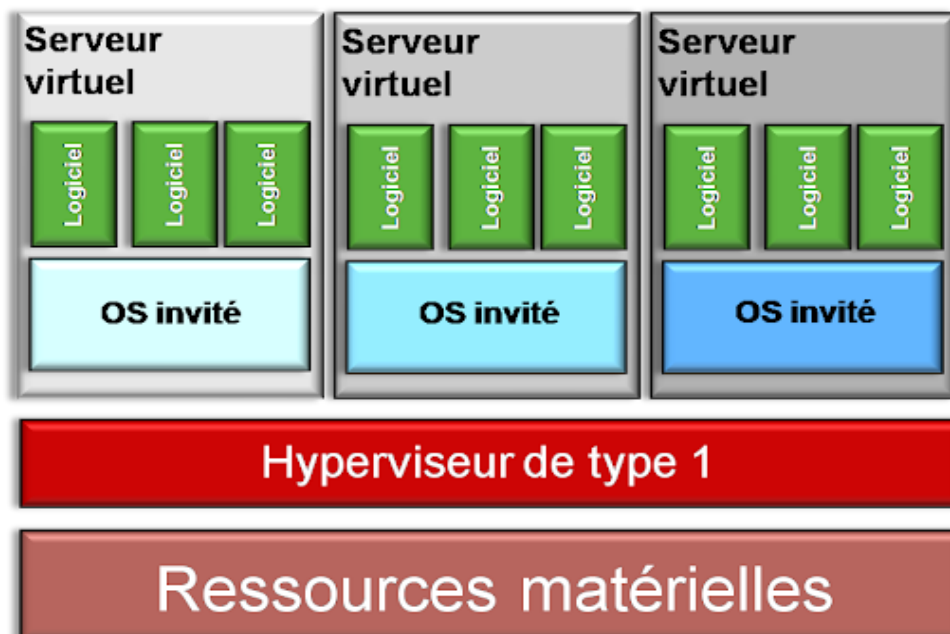


Figure 3: Hyperviseur type 1

Un **hyperviseur de type 2** est un logiciel qui s'exécute à l'intérieur d'un autre système d'exploitation. Un système d'exploitation invité s'exécutera donc en troisième niveau au-dessus du matériel. Les systèmes d'exploitation invités n'ayant pas conscience d'être virtualisés, ils n'ont pas besoin d'être adaptés. Quelques exemples de tels hyperviseurs sont VMware Workstation, VMware Fusion, l'hyperviseur open source QEMU, les produits Microsoft

Virtual PC et Virtual Server, Virtualbox d'Oracle, de même que Parallels Workstation de SWsoft et Parallels Desktop.

Le terme appel hyperviseur ou hypervisor call, ou hypercall, fait référence à l'interface de para virtualisation, par laquelle un système d'exploitation « invité » accède directement à des services à travers le logiciel de contrôle de niveau élevé (le terme superviseur fait référence au noyau du système d'exploitation qui sur les mainframes IBM. [5]

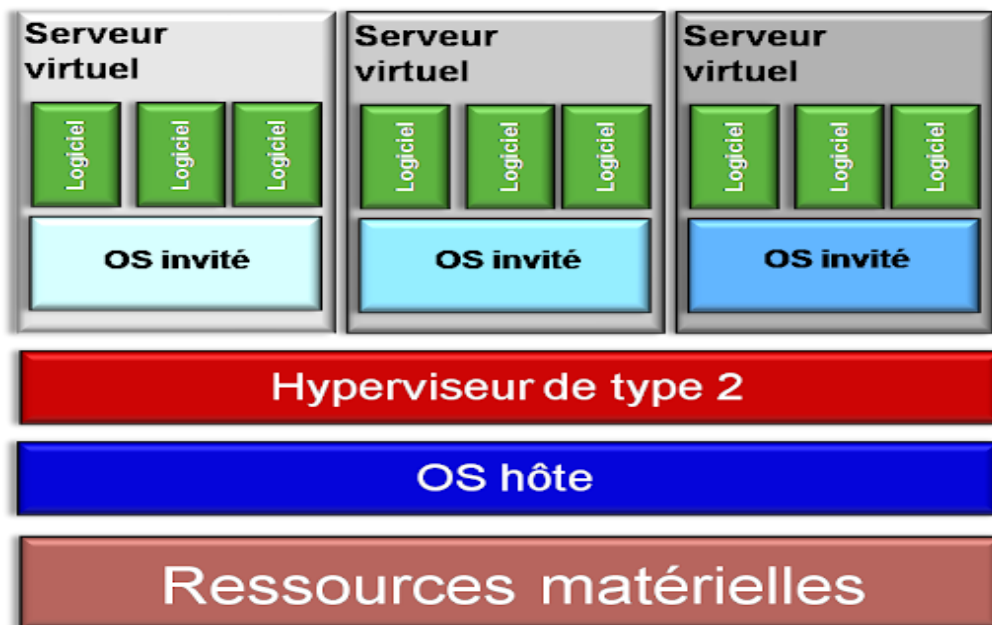


Figure 4: Hyperviseur type 2

1.4.3 Rôle de la virtualisation dans le Cloud

La virtualisation constitue l'un des piliers fondamentaux du CC . Elle permet de créer des instances virtuelles de ressources physiques (serveurs, stockage, réseaux), offrant ainsi une abstraction entre le matériel et les services déployés. Grâce à la virtualisation, plusieurs VM peuvent partager un même serveur physique, ce qui optimise l'utilisation des ressources et réduit les coûts liés à l'infrastructure.

Dans le contexte du Cloud, la virtualisation joue plusieurs rôles essentiels :

Isolation et sécurité : chaque machine virtuelle fonctionne dans un environnement isolé, limitant l'impact d'une attaque sur le reste de l'infrastructure.

Flexibilité et scalabilité : elle permet de créer, modifier ou supprimer rapidement des instances selon les besoins, facilitant ainsi l'adaptation aux variations de charge.

Optimisation des ressources : les ressources matérielles sont mieux utilisées, réduisant le gaspillage et améliorant la performance globale.

Maintenance simplifiée : les mises à jour et les sauvegardes peuvent être appliquées sur des images virtuelles, sans perturber l'ensemble du système.

Support aux services Cloud : tous les modèles de Cloud (IaaS, PaaS, SaaS) reposent sur la virtualisation pour fournir des services à distance de manière fiable et efficace.[6]

En résumé, la virtualisation transforme l'infrastructure physique en un ensemble de ressources modulables, sécurisées et faciles à gérer, ce qui constitue la base même de tout environnement Cloud moderne.

1.5 Sécurité dans le Cloud Computing

La sécurité informatique est devenue un élément essentiel dans la protection des systèmes d'information, en particulier avec l'essor du CC et de la virtualisation. Ces technologies offrent de nombreux avantages, mais exposent également les systèmes à diverses menaces et vulnérabilités. Ce chapitre présente les concepts fondamentaux de la sécurité informatique, les principales attaques ainsi que les mécanismes de protection utilisés pour sécuriser les environnements informatiques et Cloud.

Dans cette section, nous présentons les différentes techniques qui définissent la sécurité des systèmes d'information, les enjeux et les services de la sécurité. Nous décrivons les types des attaques et les méthodes de défense des systèmes d'information.

1.5.1 Enjeux de la sécurité

La sécurité informatique est l'ensemble des moyens mis en place pour empêcher la mauvaise utilisation ou la modification d'un système d'information. La sécurité a plusieurs enjeux : économiques, politiques, et juridiques .

Enjeux économiques: le système d'information est considéré comme moteur de

développement de l'entreprise. La concurrence fait que des entreprises s'investissent de plus en plus ,dans la sécurisation de leurs systèmes d'informations dans la qualité de service fournit aux clients, pour réaliser des bénéfices sur l'ensemble de leurs activités.

Enjeux politiques: la plupart des entreprises se réfèrent aux documents officiels de sécurité élaborés et recommandés par l'état. Ces documents contiennent généralement des directives qui doivent être appliquées par toute structure engagée dans un processus de sécurisation du réseau. Dans le cadre du chiffrement des données par exemple, chaque Etat définit des cadres et mesures d'utilisation des algorithmes de chiffrement et les recommande aux entreprises exerçant sur son territoire.

Le non-respect de ces mesures et recommandations peut avoir des conséquences graves sur l'entreprise. A ce niveau, l'enjeu est plus politique parce que chaque Etat souhaite être capable de décrypter toutes les informations circulant dans son espace.

Enjeux juridiques dans un réseau, nous retrouvons de l'information multiforme (numérique, papier, etc.). Le traitement de celle-ci doit se faire dans un cadre bien défini et dans le strict respect des lois en vigueur.

1.5.2 les services de la sécurité (CIA)

Les services fondamentaux de la sécurité informatique reposent sur trois piliers essentiels, connus sous l'acronyme CIA : la Confidentialité, l'Intégrité et la Disponibilité. [7]

La confidentialité garantit que l'information n'est accessible qu'aux personnes dûment autorisées, empêchant ainsi toute divulgation non autorisée des données.

L'intégrité assure que les données ne sont ni modifiées, ni altérées, ni supprimées de manière non autorisée, que ce soit lors de leur stockage ou de leur transmission.

La **disponibilité** garantit que les ressources et les informations sont accessibles aux utilisateurs autorisés au moment où ils en ont besoin, sans interruption de service.

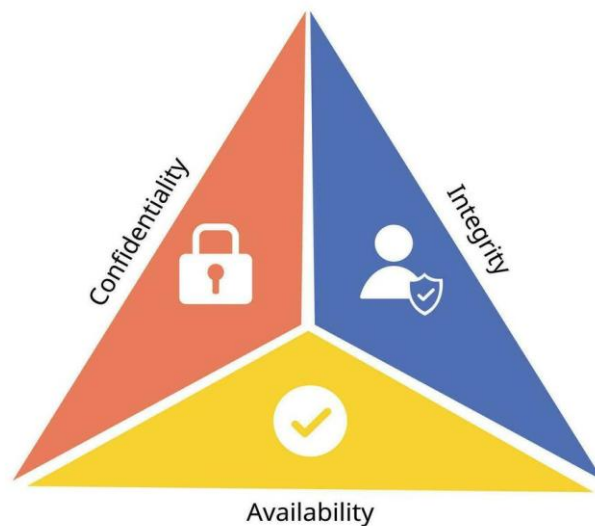


Figure 5: Les services de la sécurité (CIA)

Au-delà du modèle CIA, d'autres services complémentaires contribuent à renforcer la sécurité des systèmes d'information :[8]

- **La traçabilité** permet de conserver et d'analyser l'historique des actions effectuées sur un système, afin d'identifier l'origine d'un incident ou d'une intrusion.
- **L'authentification** vise à vérifier l'identité d'un utilisateur ou d'un système avant de lui accorder l'accès aux ressources.
- **La non-répudiation** empêche qu'une entité nie avoir effectué une action ou une communication, en fournissant une preuve irréfutable de celle-ci.

1.5.3 Les mécanismes de sécurité

Face aux menaces croissantes provenant des logiciels malveillants et des attaques informatiques, il est indispensable de mettre en place des mécanismes de sécurité afin d'assurer la confidentialité, l'intégrité et la disponibilité des systèmes d'information.

a) Pare-feu (firewall)

Un pare-feu est un ensemble de composants matériels (physiques) et logiciels (logiques) permettant de contrôler le trafic entrant et sortant d'un réseau selon une politique de sécurité prédéfinie.[9]

Le fonctionnement d'un pare-feu repose principalement sur des règles de filtrage qui définissent les adresses IP, les ports et les protocoles autorisés à communiquer avec les machines du réseau. Il agit ainsi comme une passerelle filtrante entre le réseau interne et l'extérieur.

Le pare-feu permet, d'une part, de bloquer les connexions non autorisées ou suspectes afin de protéger le réseau interne contre les attaques. D'autre part, il permet également d'éviter la fuite non contrôlée d'informations vers l'extérieur.

Il existe plusieurs types de pare-feu, qu'ils soient matériels ou logiciels, réseaux ou applicatifs. Parmi les solutions commerciales les plus connues, on peut citer Sophos ⁴et Fortigate⁵. Par ailleurs, il existe également des solutions gratuites et open source telles que pfSense⁶, Endian⁷ et IPCop⁸, qui offrent un contrôle efficace du trafic entrant et sortant grâce à des règles de filtrage

b) VPN (Virtual Private Network)

Dans les réseaux informatiques, un réseau privé virtuel (VPN) est une technologie permettant à des utilisateurs distants de communiquer de manière sécurisée à travers un réseau public, tel qu'Internet.

Le VPN repose sur des protocoles de tunnelisation qui assurent la confidentialité et l'intégrité des données échangées grâce à des algorithmes de cryptographie. Plusieurs protocoles peuvent être utilisés dans une connexion VPN, notamment : *PPTP*, *L2TP*, *IPSec*, *MPLS* et *SSL/TLS*.

c) IDS (Intrusion Detection System) / IPS (Intrusion Prevention)

Un système de détection d'intrusion (IDS) est un mécanisme destiné à détecter les activités anormales ou suspectes sur un réseau ou un hôte. Il analyse le trafic et les journaux (logs) afin d'identifier des signatures connues d'attaques.

⁴ <https://www.sophos.com/en-us.aspx>

⁵ <https://www.fortinet.com/products/next-generation-firewall.html>

⁶ <https://www.pfsense.org/download/>

⁷ <https://www.endian.com/community/download/>

⁸ <https://sourceforge.net/projects/ipcop/>

Le responsable de la sécurité des systèmes d'information (RSSI) peut ainsi surveiller les tentatives d'intrusion réussies ou échouées. Snort⁹ est un exemple largement utilisé de système IDS.[10]

système de prévention d'intrusion (IPS) est similaire à un IDS, à la différence qu'il est capable de réagir automatiquement face à une attaque détectée. Il s'agit d'un IDS actif qui peut, par exemple, bloquer des ports, des adresses IP ou interrompre une connexion suspecte.

Le **Firepower NGIPS**¹⁰ de Cisco est un exemple de solution IPS matérielle de nouvelle génération. Il existe également des solutions IPS logicielles telles que **SolarWinds Security Event Manager**¹¹, **Security Onion** et **WinPatrol**¹². [11]

Dans cette section, nous définissons les notions de menaces, de risques et de vulnérabilités en matière de sécurité informatique, puis nous présentons les principaux types de logiciels malveillants existants de nos jours.

1.6 Les cybermenaces ciblant le cloud

1.6.1 Menaces et vulnérabilités

Les menaces peuvent être classées en deux catégories principales selon leur impact sur le système d'information : les menaces passives et les menaces actives .

- **Les menaces passives:** consistent essentiellement à écouter, intercepter ou copier les informations circulant sur le réseau sans les modifier. Elles portent principalement atteinte à la confidentialité des données. Dans ce cas, l'attaquant n'altère pas l'information, mais en obtient une copie de manière non autorisée.
- **Les menaces actives:** quant à elles, ont pour objectif de perturber le fonctionnement normal du système ou du réseau. Elles peuvent entraîner la modification, la suppression ou l'altération des données, voire une interruption des services.

⁹ <https://www.cisco.com/c/en/us/products/security/ngips/index.html>

¹⁰ <https://www.cisco.com/c/en/us/products/security/ngips/index.html>

¹¹ <https://www.solarwinds.com/security-event-manager>

¹² <https://securityonion.net/>

- Le risque en sécurité informatique est évalué en fonction de deux critères principaux : la vulnérabilité et la sensibilité.

La vulnérabilité désigne le degré d'exposition d'un système ou d'un composant à une menace. Un élément peut être fortement vulnérable tout en ayant un faible niveau de sensibilité.

La sensibilité correspond à l'importance et au caractère critique des données ou des ressources concernées, notamment lorsqu'il s'agit de données confidentielles ou stratégiques.

les vulnérabilités peuvent se présenter sous plusieurs formes :

- **Vulnérabilités humaines** : l'être humain constitue souvent le maillon faible de la sécurité. Ces vulnérabilités résultent généralement d'erreurs telles que la négligence, le manque de formation, une mauvaise configuration ou une utilisation inappropriée des systèmes.
- **Vulnérabilités technologiques** : elles sont généralement dues à des failles dans la conception, le développement ou la configuration des systèmes informatiques. Avec l'évolution rapide des technologies, de nouvelles vulnérabilités sont régulièrement découvertes et publiées.
- **Vulnérabilités organisationnelles** : elles sont liées à l'absence de politiques de sécurité claires, de procédures formelles ou de règles de gestion adaptées pour faire face aux incidents de sécurité.
- **Vulnérabilités liées à la mise en œuvre** : elles apparaissent lorsque certains aspects de la sécurité ne sont pas pris en compte lors de la réalisation ou du déploiement d'un projet informatique.

1.6.2 le déni de service (Dos/DDoS)

Les attaques par déni de **service (DoS)** visent à saturer un serveur ou un routeur en l'inondant de requêtes massives, jusqu'à le rendre indisponible ou déconnecté. Bien que ces attaques soient relativement simples à mettre en place, elles sont en revanche très difficiles à contrer efficacement.[12]

Dans leur version distribuée (DDoS), ces attaques mobilisent simultanément plusieurs machines compromises (appelées botnets) pour amplifier l'impact sur la cible.

Les motivations qui poussent un attaquant à lancer ce type d'attaque sont multiples :

- **Récupérer un accès** : une attaque DoS est souvent utilisée comme première étape pour obtenir le contrôle d'une machine ou d'un réseau. Par exemple, l'attaque de type SYN Flood est fréquemment combinée à une tentative d'usurpation d'identité (IP Spoofing).
- **Masquer les traces** : en détruisant ou rendant inaccessible une station susceptible de contenir des journaux d'activité (fichiers logs), l'attaquant efface les preuves de son passage et assure ainsi sa discrétion.
- **Se venger** : ces attaques sont parfois motivées par une vengeance personnelle contre un individu, un administrateur système ou une entreprise.

1.6.3 L'usurpation d'adresse (IP Spoofing)

En se faisant passer pour un hôte de confiance, cette technique permet de s'infiltrer dans un ordinateur. Une station envoie un paquet dont l'adresse IP est autorisée par le serveur visé. La source IP envoyée trompe donc la cible qui accorde l'accès en pensant avoir affaire à une machine de confiance. Notons qu'il existe différents types de spoofing comme IP spoofing, DNS spoofing, Web spoofing...

1.6.4 Attaque d'un site web (SQLI ,XSS)

L'injection **SQL** est une technique très connue qui pourrait détruire la base de données. Elle est l'une des techniques de piratage Web les plus courantes. Elle se produit généralement lorsqu'un utilisateur est demandé de donner une entrée, telle que son nom d'utilisateur / id utilisateur, et au lieu d'un nom / id, l'utilisateur, il donne une instruction SQL qui s'exécute sans le savoir sur la base de données. [13]

Et voici des exemples :

```
txtUserId = getRequestString ("UserId");
```

```
txtSQL = "SELECT * FROM Utilisateurs WHERE UserId =" + txtUserId;
```

L'injection SQL basée sur 1 = 1 est toujours vraie

```
SELECT * FROM Utilisateurs WHERE UserId = 105 OR 1 = 1;
```

Les attaques de type **XSS** (Cross-Site Scripting) sont un type d'injection dans lequel des scripts malveillants sont injectés sur des sites Web dignes de confiance et sans danger. Les attaques XSS se produisent lorsqu'un attaquant utilise une application *Web* pour envoyer du code malveillant généralement sous la forme d'un script côté navigateur, à un utilisateur final. Le serveur Web est un programme qui héberge des sites *Web* basé à la fois sur le matériel et les logiciels. Il fournit des fichiers et autres contenus sur le site *Web* via HTTP

Les services *Web* sont devenus une partie importante de l'internet. Ils sont utilisés pour la création de fichiers, la communication, etc. Les risques sont: une configuration par défaut, des autorisations des services inutiles, des certifications *SSL* mal configurées ... et les attaques peuvent être *DoS*, *DDoS*, homme au milieu, et le hameçonnage.

1.6.5 Méthodologie d'une attaque

La méthodologie de l'attaque passe par les étapes suivants: elle commence par **la collecte des informations, le balayage du réseau, le repérage des failles**, puis **l'intrusion et l'extension des privilèges** et se termine par **le nettoyage des traces**.

Cette méthodologie n'est pas toujours appliquée telle qu'elle est, par exemple, une attaque *DoS* peut ne pas se terminer par le nettoyage des traces.

1.7 Méthodes de défense

Certaines méthodes de défense permettent de prévenir les attaques en amont, tandis que d'autres se limitent à leur détection après coup. Les principales méthodes utilisées sont les suivantes :

Le chiffrement (cryptage) : le chiffrement est un procédé cryptographique qui consiste à transformer des données lisibles en une forme incompréhensible pour toute personne ne disposant pas de la clé de déchiffrement. Il permet ainsi de se protéger contre les interceptions et les modifications non autorisées.

Au-delà de la confidentialité, le chiffrement contribue également à garantir un certain niveau d'intégrité des données, dans la mesure où des données chiffrées sont difficiles à modifier de manière cohérente sans la clé appropriée. Il constitue l'un des outils fondamentaux

de la sécurité informatique, bien qu'il ne suffise pas à lui seul à résoudre l'ensemble des problèmes de sécurité.

Contrôle logiciel (software) : afin de limiter les tentatives d'attaques, les programmes doivent être conçus de manière sécurisée dès la phase de développement. Cela inclut la validation des entrées, la gestion des droits d'accès et l'application du principe du moindre privilège.

Contrôle matériel (hardware) : il existe de nombreux dispositifs physiques contribuant à la sécurité des systèmes, tels que les modules de chiffrement matériel, les contrôleurs d'accès aux disques et les systèmes de vérification d'identité biométrique.

Cadre juridique : bien que les lois relatives à la cybercriminalité soient encore en cours de consolidation dans de nombreux pays, elles constituent un levier dissuasif important. Des organisations internationales œuvrent à l'établissement de standards éthiques et légaux encadrant l'usage des systèmes informatiques.

Dans ce chapitre, nous avons présenté les concepts fondamentaux du CC, de la virtualisation ainsi que les principaux mécanismes de sécurité et cybermenaces ciblant les environnements Cloud. Cette étude théorique constitue une base essentielle pour la mise en œuvre pratique de notre infrastructure Cloud sécurisée présentée dans le chapitre suivant

Chapitre 2 : Conception de l'architecture et présentation des outils

Ce chapitre présente le cadre conceptuel de notre projet. Nous exposons le contexte et la justification du travail réalisé, analysons les besoins fonctionnels et non-fonctionnels, décrivons l'architecture cible proposée, et justifions le choix des outils technologiques retenus. La réalisation technique et la configuration de cet environnement feront l'objet du chapitre suivant.

2.1 Contexte et Justification du Projet

2.1.1 Motivation du projet

L'adoption massive du CC par les entreprises et les organisations s'explique par ses nombreux avantages : flexibilité, réduction des coûts, accessibilité à distance et facilité de gestion des ressources informatiques. Cependant, cette évolution s'accompagne également de nouveaux défis en matière de sécurité. Les environnements Cloud virtualisés sont exposés à des menaces variées, telles que les attaques par déni de service, les intrusions non autorisées, les fuites de données ou les erreurs de configuration, qui peuvent compromettre la confidentialité, l'intégrité et la disponibilité des données.

Dans ce contexte, notre projet prend tout son sens. La motivation principale est de comprendre ces enjeux et de proposer des solutions concrètes permettant de sécuriser un environnement Cloud virtualisé. En analysant les risques, en identifiant les vulnérabilités et en mettant en place des mécanismes de protection adaptés, ce projet vise à fournir un cadre pratique et sécurisé pour l'exploitation d'un Cloud, tout en garantissant la continuité et la fiabilité des services.

Ainsi, ce projet n'est pas seulement théorique : il permet de relier les concepts étudiés en sécurité informatique et en virtualisation à des applications pratiques, préparant le terrain pour la mise en place concrète de l'environnement Cloud sécurisé dans le chapitre suivant.

2.1.2 Problématique

Avec la généralisation du CC, les environnements virtualisés sont exposés à de multiples cyberattaques pouvant compromettre la sécurité des données et la disponibilité des services.

Des services couramment déployés dans ces environnements présentent des vulnérabilités exploitables :

- Le service Secure Shell (SSH) est vulnérable aux attaques par force brute visant à deviner les identifiants de connexion.
- Le serveur web Apache peut présenter des configurations incorrectes et des versions obsolètes exploitables.
- La base de données MySQL contient des données sensibles pouvant être ciblées par des intrusions.

La problématique de notre projet consiste donc à répondre à la question suivante :

"Comment concevoir et sécuriser un environnement Cloud IaaS virtualisé afin de détecter et contrer efficacement les cyberattaques, tout en garantissant la confidentialité, l'intégrité et la disponibilité des données ?"

2.2 Analyse des besoins et spécifications

2.2.1 Besoins fonctionnels

Notre projet doit répondre aux besoins fonctionnels suivants :

- Simuler des cyberattaques réalistes (reconnaissance, scan de vulnérabilités, force brute SSH) contre un environnement Cloud IaaS.
- Centraliser et analyser les journaux (logs) provenant des machines cibles en temps réel.
- Détecter automatiquement les intrusions et générer des alertes critiques classifiées selon le framework MITRE ATT&CK.
- Alerter instantanément l'équipe de sécurité via une notification Telegram dès qu'une alerte critique est déclenchée.

2.2.2 Besoins non-fonctionnels

Notre projet doit également satisfaire les contraintes suivantes :

- **Sécurité** : les communications entre les trois machines de l'équipe doivent être chiffrées de bout en bout via un réseau virtuel privé (ZeroTier One).

- **Coût** : utilisation exclusive d'outils open source et gratuits (Wazuh, VirtualBox, ZeroTier, PFSense, Docker).
- **Portabilité** : le déploiement des outils doit être facilité par la conteneurisation (Docker & Docker-Compose).
- **Réactivité** : le système doit détecter et notifier les attaques en temps réel, sans nécessiter une surveillance permanente du Dashboard.

2.3 Architecture cible proposée

L'architecture conceptuelle de notre projet repose sur la séparation des rôles en trois entités distinctes, interconnectées de manière sécurisée via ZeroTier One, formant ainsi un Cloud privé IaaS distribué.

Zone offensive (Red Team) : Représentée par une machine Kali Linux dédiée aux tests de pénétration, depuis laquelle sont simulées les cyberattaques (Nmap, Nikto, Hydra).

Zone cible (Victime / Blue Team) : Un serveur Ubuntu hébergeant les services exposés (Apache, MySQL, SSH), le pare-feu PFSense pour le filtrage du trafic réseau, ainsi que l'agent Wazuh chargé de collecter les logs et de les transmettre au **Manager**.

Zone de supervision (SOC) : Un serveur central hébergeant le Wazuh Manager via Docker, assurant la corrélation des événements, la génération des alertes et l'envoi des notifications via le script Python Telegram.

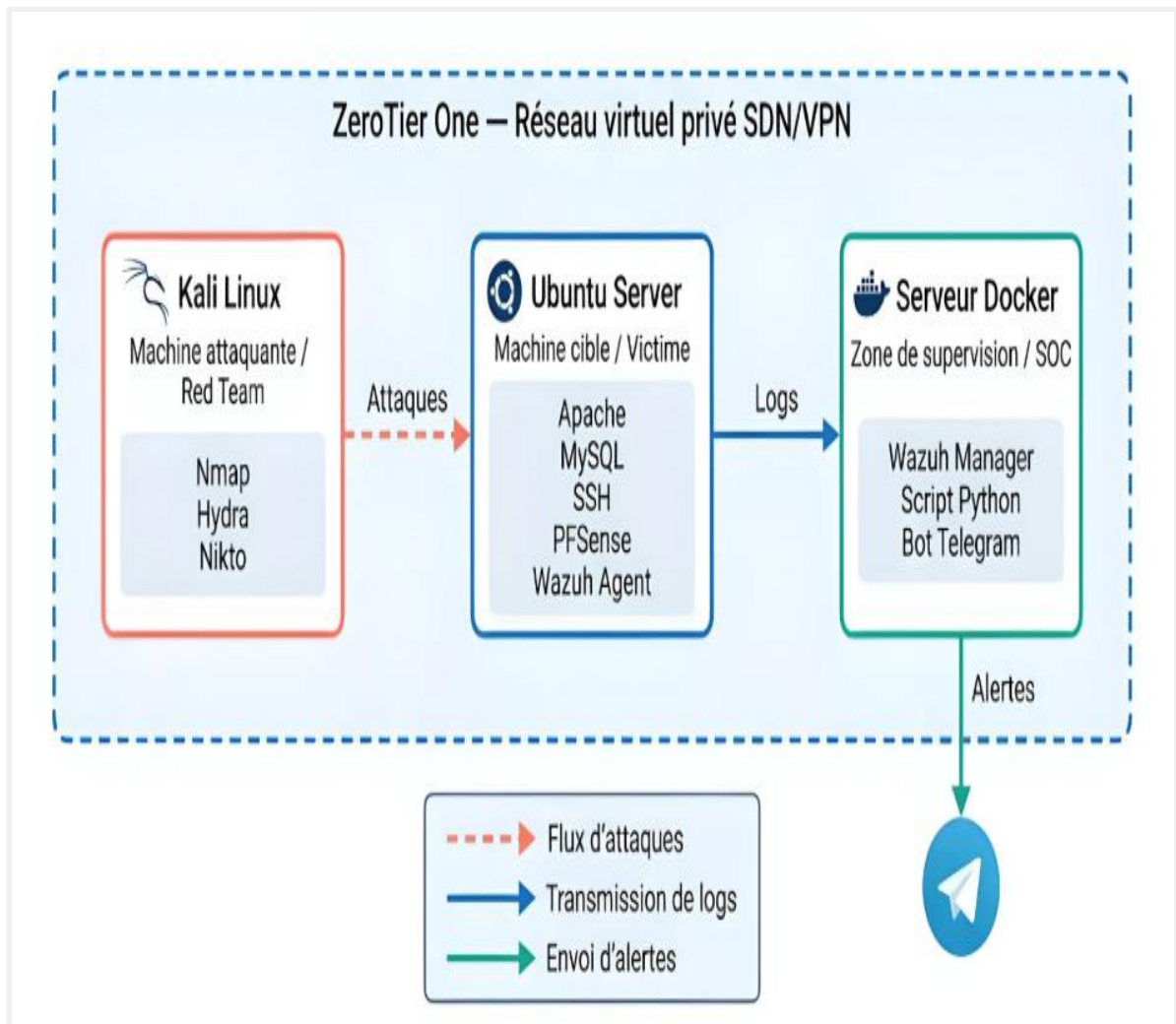


Figure 6: Architecture du projet

Ce schéma représente l'architecture technique et le flux de données mis en œuvre dans notre projet.

Elle se compose de trois zones interconnectées de manière sécurisée à travers le réseau **virtuel ZeroTier One** : La zone offensive (Red Team) : Portée par Kali Linux pour simuler les attaques (Nmap, Hydra, Nikto).

La zone cible (Victime) : Un serveur Ubuntu Server sécurisé par un pare-feu pfSense, hébergeant l'agent Wazuh qui collecte les logs en temps réel.

La zone de supervision (SOC) : Le manager Wazuh déployé sous Docker, qui corrèle les événements et utilise un script Python personnalisé pour envoyer des alertes instantanées vers un bot Telegram.

2.4 Étude comparative et choix des outils

2.4.1 Environnement de virtualisation et réseau



Figure 7: Logo VirtualBox

VirtualBox¹³ VirtualBox est un hyperviseur de type 2 développé par Oracle, permettant de créer et gérer des (VM) sur un PC standard. Il offre une interface intuitive et supporte de nombreux systèmes d'exploitation invités.

- Dans notre projet, VirtualBox est utilisé sur les PC des membres de l'équipe pour héberger les VM Ubuntu Server et Kali Linux, simulant ainsi l'infrastructure d'un Cloud IaaS.



Figure 8: Logo ZeroTier One

ZeroTier One¹⁴ est une solution de réseau défini par logiciel (SDN) permettant de créer un réseau virtuel privé entre machines distantes, indépendamment de leur localisation géographique. Chaque machine reçoit une adresse IP virtuelle fixe et communique avec les autres comme sur un réseau local.

- Dans notre projet, ZeroTier One joue un rôle central : il interconnecte les trois PC des membres de l'équipe travaillant à distance, simulant ainsi un Cloud privé distribué réaliste, sans nécessiter de configuration réseau complexe.

2.4.2 Plateforme offensive (Red Team)



Figure 9: Logo Kali Linux

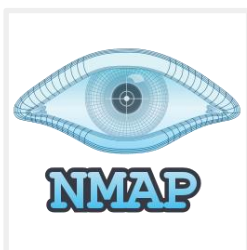
Kali Linux¹⁵ est une distribution Linux spécialisée en cybersécurité offensive, développée par Offensive Security. Elle embarque nativement des centaines d'outils dédiés aux tests d'intrusion et à l'évaluation de la sécurité des systèmes.

- Dans notre projet, Kali Linux constitue la machine attaquante (Red Team). Elle intègre les trois outils utilisés dans nos scénarios d'attaques :

¹³ <https://www.virtualbox.org/>

¹⁴ <https://www.zerotier.com/>

¹⁵ <https://www.kali.org/>



Nmap (Network Mapper) ¹⁶: outil de cartographie réseau permettant de découvrir les hôtes actifs, les ports ouverts et les versions des services en cours d'exécution sur la machine cible. Il constitue la phase de reconnaissance de notre simulation.

Figure 10: Logo Nmap



Nikto : ¹⁷scanner de vulnérabilités web open source permettant d'auditer un serveur web et d'identifier les configurations incorrectes, les versions obsolètes et les failles de sécurité connues. Il est utilisé pour analyser le serveur Apache de la machine cible.

Figure 11: Logo Nikto



Hydra : outil de force brute multi-protocoles permettant de tester automatiquement des milliers de mots de passe contre un service cible. Dans notre projet, il est utilisé pour simuler une attaque par force brute sur le service SSH de la machine Ubuntu.

Figure 12:: Logo Hydra

2.4.3 Plateforme cible et services applicatifs



Figure 13: Logo Ubuntu Server

Ubuntu Server ¹⁸est une distribution Linux open source développée par Canonical, reconnue pour sa stabilité, sa sécurité et sa large communauté de support. Elle est conçue spécifiquement pour les environnements serveurs.

- Dans notre projet, Ubuntu Server joue le rôle de machine cible (victime). Elle héberge le serveur web Apache, la base de données MySQL, le service SSH et l'agent Wazuh chargé de collecter les logs.

¹⁶ <https://seela.io/cybertraining/cours-complet-scan-reseau-nmap/>

¹⁷ <https://medium.com/@amaltomparakkaden/nikto-a-comprehensive-web-vulnerability-scanner-bb9f0ec105e4>

¹⁸ : <https://ubuntu.com/server>



Figure 14: Logo Apache & MySQL

Apache¹⁹ est un serveur web open source, l'un des plus utilisés au monde, permettant d'héberger des applications web et de servir des contenus HTTP.

MySQL²⁰ est un système de gestion de bases de données relationnelles open source, largement adopté pour le stockage de données applicatives.

- Dans notre projet, Apache et MySQL sont installés sur Ubuntu Server pour simuler un environnement web réaliste exposé aux attaques. Ils constituent les cibles principales des scénarios de test.



Figure 15: Logo PFSense

PFSense²¹ est un pare-feu open source basé sur FreeBSD, offrant des fonctionnalités avancées de filtrage du trafic réseau, de gestion des règles de sécurité et de surveillance du réseau.

- Dans notre projet, PFSense constitue la première ligne de défense de notre environnement Cloud. Il est installé sur la machine Ubuntu Server pour filtrer le trafic entrant et sortant selon des règles définies.

2.4.4 Solution de supervision et d'alerte (Blue Team)



Figure 16: Logo Wazuh

Wazuh²² est une plateforme open source de sécurité (SIEM — Security Information and Event Management) assurant la collecte, l'analyse et la corrélation des logs en temps réel. Elle intègre nativement le framework MITRE ATT&CK pour la classification des menaces détectées.

- Dans notre projet, Wazuh constitue le cœur de notre système de détection. Il repose sur deux composants :
 - **Wazuh Manager** : serveur central déployé via Docker sur la machine de supervision. Il reçoit, analyse et corrèle les événements de sécurité et génère les alertes critiques.
 - **Wazuh Agent** : installé sur la machine Ubuntu Server cible. Il collecte les logs système en temps réel (auth.log, access.log) et les transmet au Manager pour analyse.

¹⁹ <https://httpd.apache.org/>

²⁰ <https://www.mysql.com/>

²¹ <https://www.pfsense.org/>

²² <https://www.pfsense.org/>



Figure 17: Logo Docker

Docker²³ & Docker-Compose Docker est une plateforme de conteneurisation permettant de déployer des applications sous forme de conteneurs isolés, légers et portables. Docker-Compose orchestre plusieurs conteneurs via un fichier de configuration YAML.

➤ Dans notre projet, Docker et Docker-Compose sont utilisés pour déployer le Wazuh Manager de manière moderne, reproductible et isolée, conformément aux pratiques DevSecOps et Cloud-Native.



Figure 18: Logo Telegram

Script Python & API Telegram Pour pallier la nécessité d'une surveillance permanente du Dashboard Wazuh, nous avons développé un script Python personnalisé nommé custom-telegram, intégré directement au SIEM via le fichier de configuration ossec.conf.

➤ Dès qu'une alerte de niveau 10 ou supérieur est déclenchée par Wazuh, ce script extrait automatiquement les informations de l'alerte (type d'attaque, adresse IP source, horodatage) et les envoie instantanément via l'API Telegram à un Bot dédié, permettant à l'équipe de sécurité d'être alertée en temps réel depuis n'importe quel endroit.

Ce chapitre de conception nous a permis de définir clairement la problématique, d'analyser les besoins du projet, de présenter l'architecture cible et de justifier le choix des outils technologiques retenus. L'ensemble de ces éléments forme un environnement Cloud privé IaaS cohérent et réaliste. Le chapitre suivant sera consacré au déploiement et à la configuration technique de cet environnement, ainsi qu'aux tests d'intrusion réalisés et à l'analyse des résultats obtenus.

²³ <https://www.docker.com/>

Chapitre 3 : Mise en œuvre et Tests

Dans ce chapitre, nous présentons la phase pratique de notre projet, consacrée à la mise en œuvre de l'environnement Cloud virtualisé sécurisé ainsi qu'aux différents tests réalisés pour valider son efficacité face aux cybermenaces.

L'objectif principal de cette étape est de déployer une infrastructure distribuée simulant un environnement Cloud IaaS réel, puis d'y intégrer une solution de supervision et de détection des intrusions basée sur la plateforme Wazuh SIEM. Cette infrastructure repose sur trois machines distinctes interconnectées à travers un réseau virtuel privé sécurisé via ZeroTier One.

Après la configuration des différentes machines et le déploiement du SIEM Wazuh, nous avons développé un système d'alerting automatisé permettant l'envoi instantané de notifications critiques via Telegram. Enfin, plusieurs scénarios d'attaques réelles ont été simulés afin d'évaluer les capacités de détection, de corrélation et de supervision de notre architecture de sécurité.

3.1 Mise en place du réseau virtuel

3.1.1 Présentation de ZeroTier One

ZeroTier One est une solution de réseau défini par logiciel (SDN : Software-Defined Networking) permettant de créer des réseaux virtuels sécurisés entre machines distantes, indépendamment de leur localisation géographique. Concrètement, chaque machine rejoignant un réseau ZeroTier se voit attribuer une adresse IP virtuelle fixe, et communique avec les autres membres du réseau comme si elles étaient connectées au même réseau local (LAN).

Les avantages de ZeroTier One dans notre contexte sont les suivants :

- Connexion sécurisée chiffrée de bout en bout entre les trois PC distants
- Attribution d'adresses IP virtuelles stables à chaque machine
- Simulation réaliste d'un réseau Cloud distribué sans coût d'infrastructure
- Facilité de déploiement et de configuration (interface web intuitive)
- Compatibilité avec Windows, Linux et macOS

3.1.2 Création et configuration du réseau ZeroTier

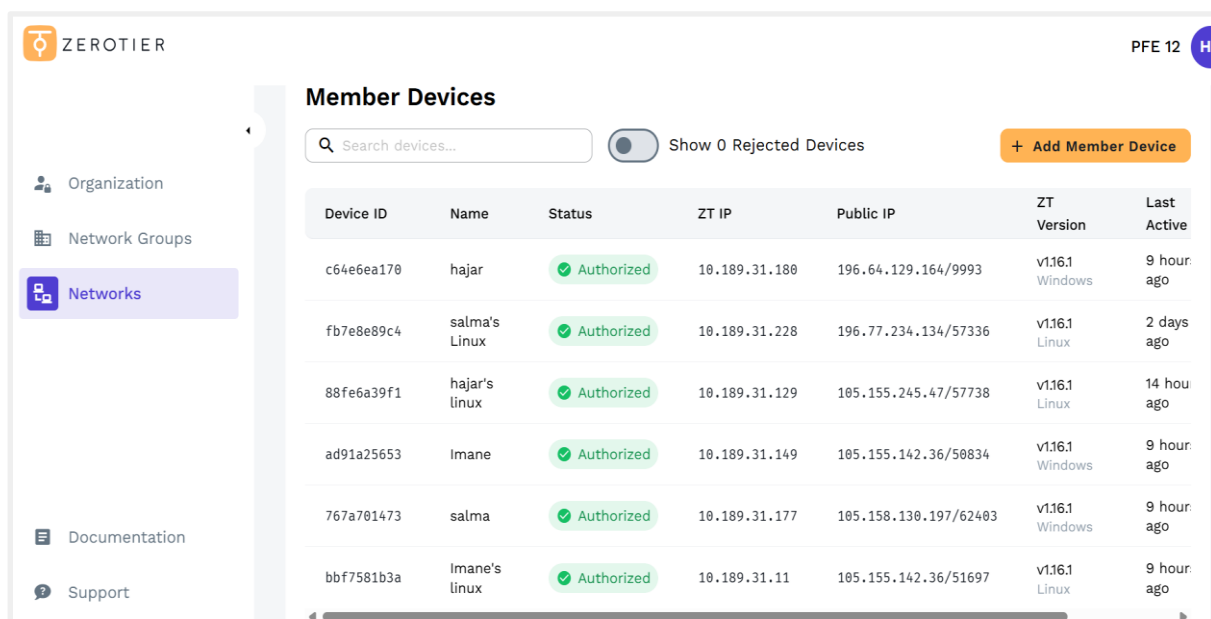
La procédure de mise en place du réseau virtuel s'est déroulée en plusieurs étapes :

Tableau 1 : Étapes de configuration de ZeroTier One

Étape	Action	Détail
1	Création du compte	Inscription sur my.zerotier.com par un membre de l'équipe
2	Création du réseau	Génération d'un Network ID unique (b9a18a606f61b45c)
3	Installation du client	Installation de ZeroTier CLI sur chaque PC de l'équipe
4	Adhésion au réseau	Commande : <code>sudo zerotier-cli join b9a18a606f61b45c</code>
5	Autorisation des membres	Approbation manuelle de chaque machine depuis le dashboard
6	Attribution IP virtuelle	Chaque PC reçoit une IP dans la plage 10.189.x.x

3.1.3 Vérification de la connectivité

Après la configuration du réseau, des tests de connectivité ont été effectués afin de vérifier la communication entre les machines.



The screenshot shows the ZeroTier dashboard interface. On the left is a sidebar with navigation links: Organization, Network Groups, Networks (highlighted), Documentation, and Support. The main area is titled 'Member Devices' and contains a search bar, a toggle for 'Show 0 Rejected Devices', and an '+ Add Member Device' button. Below this is a table listing authorized devices.

Device ID	Name	Status	ZT IP	Public IP	ZT Version	Last Active
c64e6ea170	hajar	Authorized	10.189.31.180	196.64.129.164/9993	v1.16.1 Windows	9 hour ago
fb7e8e89c4	salma's Linux	Authorized	10.189.31.228	196.77.234.134/57336	v1.16.1 Linux	2 days ago
88fe6a39f1	hajar's linux	Authorized	10.189.31.129	105.155.245.47/57738	v1.16.1 Linux	14 hour ago
ad91a25653	Imane	Authorized	10.189.31.149	105.155.142.36/50834	v1.16.1 Windows	9 hour ago
767a701473	salma	Authorized	10.189.31.177	105.158.130.197/62403	v1.16.1 Windows	9 hour ago
bbf7581b3a	Imane's linux	Authorized	10.189.31.11	105.155.142.36/51697	v1.16.1 Linux	9 hour ago

Figure 19: Tableau de bord ZeroTier One

Les commandes ping suivantes ont été utilisées :

```
(salmafadili@kali)-[~]
$ ping 10.189.31.11
PING 10.189.31.11 (10.189.31.11) 56(84) bytes of data.
64 bytes from 10.189.31.11: icmp_seq=1 ttl=64 time=57.5 ms
64 bytes from 10.189.31.11: icmp_seq=2 ttl=64 time=72.5 ms
64 bytes from 10.189.31.11: icmp_seq=3 ttl=64 time=91.9 ms
64 bytes from 10.189.31.11: icmp_seq=4 ttl=64 time=30.0 ms
64 bytes from 10.189.31.11: icmp_seq=5 ttl=64 time=30.6 ms
64 bytes from 10.189.31.11: icmp_seq=6 ttl=64 time=51.3 ms
^C
— 10.189.31.11 ping statistics —
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 29.958/55.606/91.854/21.995 ms
```

Figure 20: Test de connectivité Kali → Ubuntu

```
imane@imane-VirtualBox:~$ ping 10.189.31.129
PING 10.189.31.129 (10.189.31.129) 56(84) bytes of data.
64 bytes from 10.189.31.129: icmp_seq=1 ttl=64 time=1114 ms
64 bytes from 10.189.31.129: icmp_seq=2 ttl=64 time=113 ms
64 bytes from 10.189.31.129: icmp_seq=3 ttl=64 time=51.2 ms
64 bytes from 10.189.31.129: icmp_seq=4 ttl=64 time=39.1 ms
^C
--- 10.189.31.129 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 39.082/329.311/1113.869/453.832 ms, pipe 2
imane@imane-VirtualBox:~$
```

Figure 21: Test de connectivité Ubuntu → Wazuh

```
hajar@wazuh-manager:~/wazuh-docker/indexer-certs-creator$ ping 10.189.31.129
PING 10.189.31.129 (10.189.31.129) 56(84) bytes of data.
64 bytes from 10.189.31.129: icmp_seq=1 ttl=64 time=0.007 ms
64 bytes from 10.189.31.129: icmp_seq=2 ttl=64 time=0.005 ms
64 bytes from 10.189.31.129: icmp_seq=3 ttl=64 time=0.077 ms
64 bytes from 10.189.31.129: icmp_seq=4 ttl=64 time=0.048 ms
64 bytes from 10.189.31.129: icmp_seq=5 ttl=64 time=0.024 ms
64 bytes from 10.189.31.129: icmp_seq=6 ttl=64 time=0.043 ms
64 bytes from 10.189.31.129: icmp_seq=7 ttl=64 time=0.048 ms
64 bytes from 10.189.31.129: icmp_seq=8 ttl=64 time=0.043 ms
64 bytes from 10.189.31.129: icmp_seq=9 ttl=64 time=0.041 ms
64 bytes from 10.189.31.129: icmp_seq=10 ttl=64 time=0.044 ms
64 bytes from 10.189.31.129: icmp_seq=11 ttl=64 time=0.062 ms
64 bytes from 10.189.31.129: icmp_seq=12 ttl=64 time=0.056 ms
64 bytes from 10.189.31.129: icmp_seq=13 ttl=64 time=0.057 ms
64 bytes from 10.189.31.129: icmp_seq=14 ttl=64 time=0.044 ms
64 bytes from 10.189.31.129: icmp_seq=15 ttl=64 time=0.042 ms
64 bytes from 10.189.31.129: icmp_seq=16 ttl=64 time=0.040 ms
64 bytes from 10.189.31.129: icmp_seq=17 ttl=64 time=0.046 ms
64 bytes from 10.189.31.129: icmp_seq=18 ttl=64 time=0.042 ms
64 bytes from 10.189.31.129: icmp_seq=19 ttl=64 time=0.041 ms
64 bytes from 10.189.31.129: icmp_seq=20 ttl=64 time=0.042 ms
64 bytes from 10.189.31.129: icmp_seq=21 ttl=64 time=0.042 ms
64 bytes from 10.189.31.129: icmp_seq=22 ttl=64 time=0.044 ms
64 bytes from 10.189.31.129: icmp_seq=23 ttl=64 time=0.042 ms
64 bytes from 10.189.31.129: icmp_seq=24 ttl=64 time=0.040 ms
^C
--- 10.189.31.129 ping statistics ---
24 packets transmitted, 24 received, 0% packet loss, time 23529ms
rtt min/avg/max/mdev = 0.024/0.079/0.007/0.152 ms
hajar@wazuh-manager:~/wazuh-docker/indexer-certs-creator$
```

Figure 22: Test de connectivité Wazuh → Ubuntu

Les résultats obtenus ont confirmé la bonne interconnexion des trois machines via le réseau virtuel ZeroTier.

Cette étape est essentielle car toutes les communications du projet reposent sur cette connectivité, notamment :

- La transmission des logs du Wazuh Agent
- Les attaques lancées depuis Kali Linux
- Les notifications générées par le système de supervision.

3.2 Configuration des machines

3.2.1 Configuration de la machine Ubuntu Server (Cible)

La machine Ubuntu Server joue le rôle de machine victime dans notre architecture. Elle héberge les différents services exposés aux attaques ainsi que l'agent Wazuh chargé de la collecte des journaux système.

Configuration générale

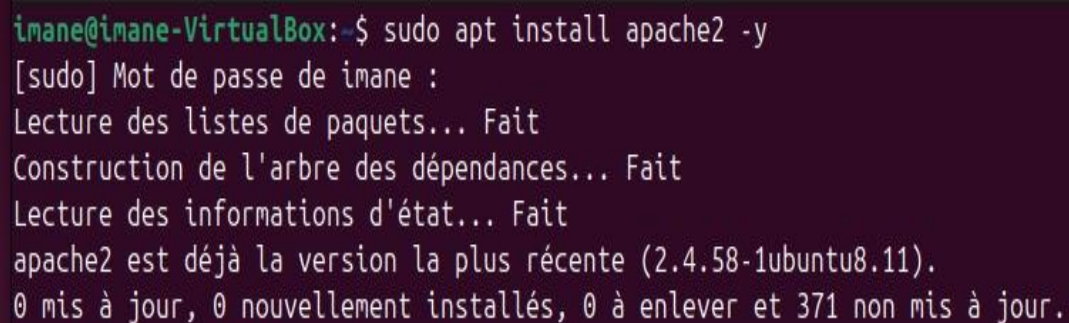
Système d'exploitation : Ubuntu Server 22.04 LTS

Adresse IP : *10.189.31.11*

Installation des services

Les services suivants ont été installés :

Installation du serveur Apache2

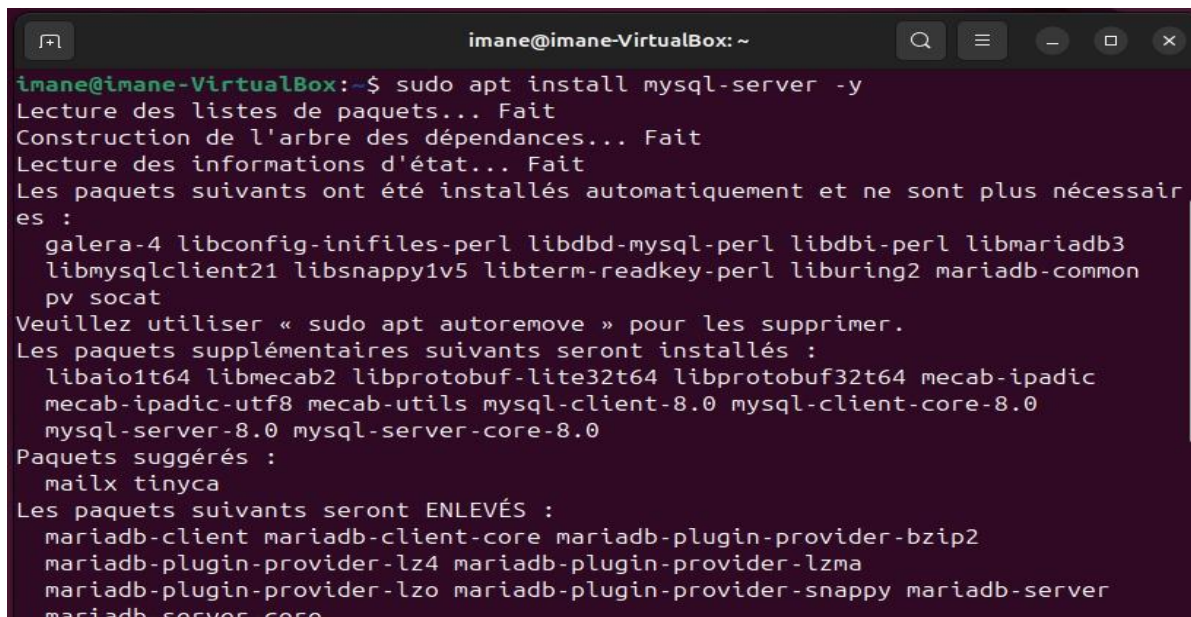


```
imane@imane-VirtualBox:~$ sudo apt install apache2 -y
[sudo] Mot de passe de imane :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
apache2 est déjà la version la plus récente (2.4.58-1ubuntu8.11).
0 mis à jour, 0 nouvellement installés, 0 à enlever et 371 non mis à jour.
```

Figure 23: Installation du serveur Apache2

Apache2 est utilisé pour héberger le serveur web cible des scans Nikto.

Installation de MySQL

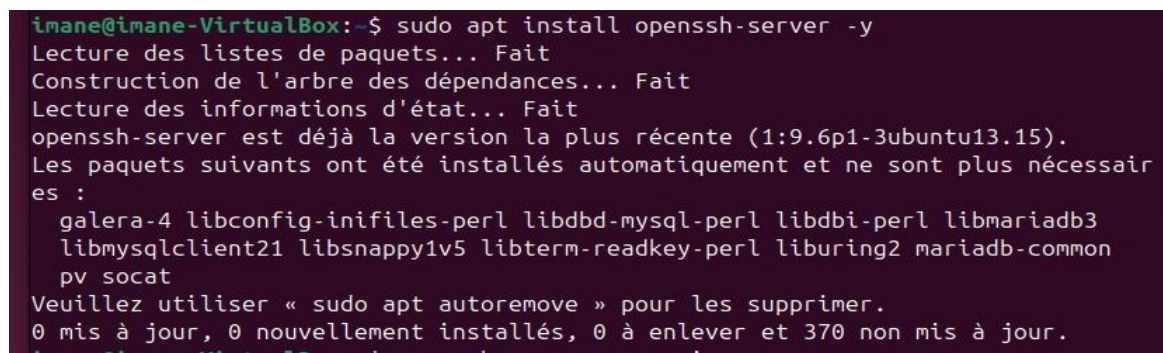


```
imane@imane-VirtualBox: ~  
imane@imane-VirtualBox:~$ sudo apt install mysql-server -y  
Lecture des listes de paquets... Fait  
Construction de l'arbre des dépendances... Fait  
Lecture des informations d'état... Fait  
Les paquets suivants ont été installés automatiquement et ne sont plus nécessair  
es :  
  galera-4 libconfig-inifiles-perl libdbd-mysql-perl libdbi-perl libmariadb3  
  libmysqlclient21 libsnappy1v5 libterm-readkey-perl liburing2 mariadb-common  
  pv socat  
Veuillez utiliser « sudo apt autoremove » pour les supprimer.  
Les paquets supplémentaires suivants seront installés :  
  libaio1t64 libmecab2 libprotobuf-lite32t64 libprotobuf32t64 mecab-ipadic  
  mecab-ipadic-utf8 mecab-utils mysql-client-8.0 mysql-client-core-8.0  
  mysql-server-8.0 mysql-server-core-8.0  
Paquets suggérés :  
  mailx tinycat  
Les paquets suivants seront ENLEVÉS :  
  mariadb-client mariadb-client-core mariadb-plugin-provider-bzip2  
  mariadb-plugin-provider-lz4 mariadb-plugin-provider-lzma  
  mariadb-plugin-provider-lzo mariadb-plugin-provider-snappy mariadb-server  
  mariadb-server-core
```

Figure 24: Installation de MySQL

permet de simuler un environnement applicatif contenant des données sensibles.

Installation du service SSH



```
imane@imane-VirtualBox:~$ sudo apt install openssh-server -y  
Lecture des listes de paquets... Fait  
Construction de l'arbre des dépendances... Fait  
Lecture des informations d'état... Fait  
openssh-server est déjà la version la plus récente (1:9.6p1-3ubuntu13.15).  
Les paquets suivants ont été installés automatiquement et ne sont plus nécessair  
es :  
  galera-4 libconfig-inifiles-perl libdbd-mysql-perl libdbi-perl libmariadb3  
  libmysqlclient21 libsnappy1v5 libterm-readkey-perl liburing2 mariadb-common  
  pv socat  
Veuillez utiliser « sudo apt autoremove » pour les supprimer.  
0 mis à jour, 0 nouvellement installés, 0 à enlever et 370 non mis à jour.
```

Figure 25: Installation et activation du service SSH sur Ubuntu Server

Le service SSH constitue la cible principale des attaques par force brute réalisées avec Hydra.

3.2.2. Configuration et Filtrage via le Pare-feu pfSense

Afin de garantir une sécurité périmétrique rigoureuse et de limiter les connexions non autorisées vers notre serveur de production, nous avons implémenté **pfSense**, une solution open-source de pare-feu et de routage avancée. Contrairement à un simple pare-feu local,

pfSense agit comme une passerelle filtrante dédiée, permettant de segmenter le trafic et d'appliquer des règles de filtrage strictes basées sur les adresses IP sources, les ports et les protocoles.

La politique de sécurité appliquée sur pfSense repose sur le principe du moindre privilège (Tout ce qui n'est pas explicitement autorisé est bloqué par défaut). Les ports nécessaires au fonctionnement de notre infrastructure ont été configurés et autorisés comme suit :

- **Port 22/TCP (SSH) :** Autorisé uniquement pour permettre l'administration à distance et simuler l'attaque par force brute.
- **Port 80/TCP (HTTP) :** Autorisé pour donner accès au serveur Web Apache2 et tester le scan de vulnérabilités.
- **Port 3306/TCP (MySQL) :** Autorisé pour les flux liés à la base de données.
- **Ports de communication ZeroTier et Agent Wazuh :** Autorisés pour permettre l'interconnexion chiffrée et la remontée transparente des journaux d'événements (logs).

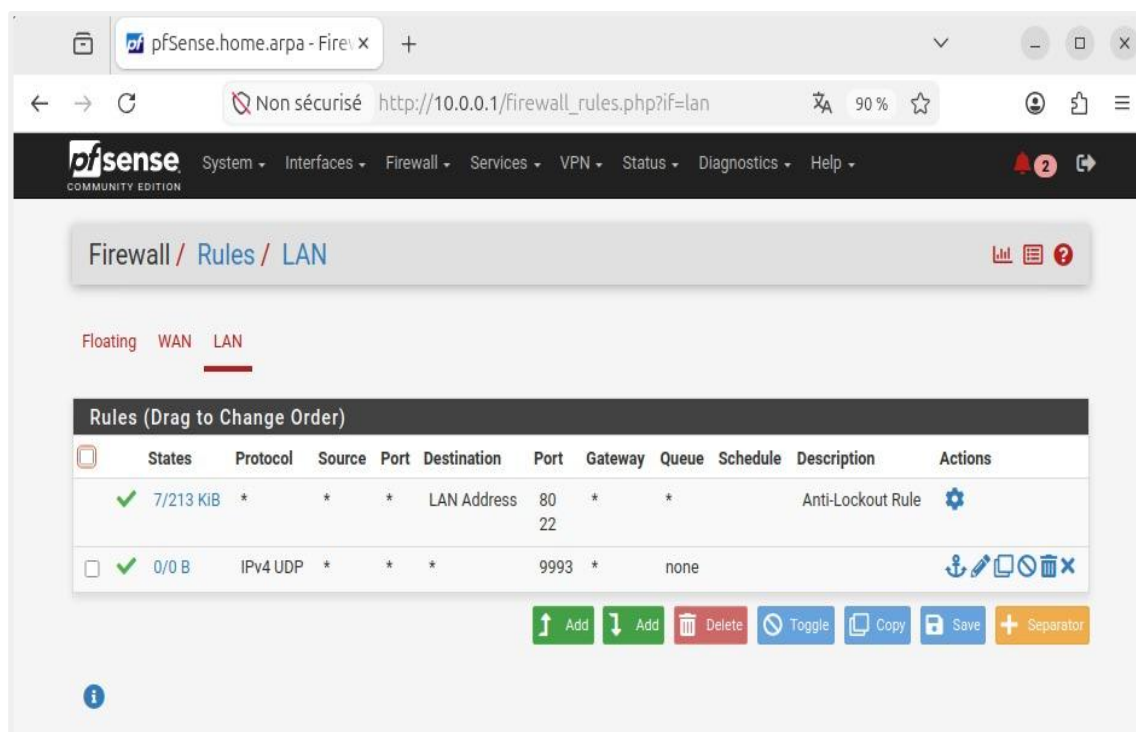


Figure 26: Configuration pfSense (règles)

3.2.3. Installation et Configuration de l'Agent Wazuh

Pour assurer une surveillance continue et centralisée de notre serveur de production sous Ubuntu Server, nous avons procédé à l'installation opérationnelle du paquet **wazuh-agent**. En tant que composant de détection déporté (HIDS), cet agent s'exécute en arrière-plan avec des privilèges système afin de superviser l'état de sécurité interne de la machine hôte.

Le rôle principal de cet agent s'articule autour de trois missions fondamentales :

- **La collecte et la centralisation des logs** : L'agent intercepte en temps réel les lignes de journaux générées par le système d'exploitation et les applications natives.
- **La surveillance de l'intégrité des fichiers (FIM)** : Il remonte toute modification suspecte ou accès non autorisé aux fichiers de configuration critiques du serveur.
- **La transmission chiffrée et sécurisée** : Toutes les données collectées sont normalisées, puis acheminées de manière transparente vers le Wazuh Manager via un protocole chiffré basé sur TLS, évitant ainsi toute interception ou altération des logs sur le réseau virtuel.

Afin de valider nos scénarios d'attaques, l'agent a été configuré spécifiquement pour surveiller et analyser en continu les fichiers de logs stratégiques suivants :

- **/var/log/auth.log** : Fichier crucial pour la sécurité du système, enregistrant toutes les tentatives d'authentification et les sessions d'audit (utilisé pour corréliser et détecter l'attaque Brute Force SSH via Hydra).
- **/var/log/apache2/access.log** : Journal d'accès du serveur Web, répertoriant l'ensemble des requêtes HTTP entrantes (utilisé pour identifier les requêtes malformées et les scans automatisés de vulnérabilités via Nikto).
- **/var/log/syslog** : Journal centralisateur du système d'exploitation Linux, consignait les messages d'information, les alertes de sécurité globales et les événements liés aux démons du serveur.

```
imane@imane-VirtualBox: ~  
imane@imane-VirtualBox:~$ sudo systemctl status wazuh-agent  
● wazuh-agent.service - Wazuh agent  
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; pres>  
   Active: active (running) since Sat 2026-05-23 22:31:08 +01; 19min ago  
     Tasks: 27 (limit: 2267)  
    Memory: 7.9M (peak: 54.8M swap: 9.2M swap peak: 9.2M)  
       CPU: 5.987s  
    CGroup: /system.slice/wazuh-agent.service  
            └─1470 /var/ossec/bin/wazuh-execd  
              └─1509 /var/ossec/bin/wazuh-agentd  
                └─1535 /var/ossec/bin/wazuh-syscheckd  
                  └─1563 /var/ossec/bin/wazuh-logcollector  
                    └─1571 /var/ossec/bin/wazuh-modulesd  
  
mai 23 22:30:52 imane-VirtualBox env[1242]: Deleting PID file '/var/ossec/var/r>  
mai 23 22:30:53 imane-VirtualBox env[1242]: Deleting PID file '/var/ossec/var/r>  
mai 23 22:30:54 imane-VirtualBox env[1242]: Deleting PID file '/var/ossec/var/r>  
mai 23 22:30:58 imane-VirtualBox env[1242]: Started wazuh-execd...  
mai 23 22:31:01 imane-VirtualBox env[1242]: Started wazuh-agentd...  
mai 23 22:31:03 imane-VirtualBox env[1242]: Started wazuh-syscheckd...  
mai 23 22:31:04 imane-VirtualBox env[1242]: Started wazuh-logcollector...  
mai 23 22:31:06 imane-VirtualBox env[1242]: Started wazuh-modulesd...  
mai 23 22:31:08 imane-VirtualBox env[1242]: Completed.  
mai 23 22:31:08 imane-VirtualBox systemd[1]: Started wazuh-agent.service - Wazu>  
lines 1-23...skipping...
```

Figure 27: État service wazuh-agent

3.2.4 Configuration de Kali Linux (Machine attaquante)

La machine Kali Linux constitue la plateforme offensive du projet (Red Team).

Configuration générale

- Système : Kali Linux Rolling
- Adresse IP : 10.189.31.228

Cette machine contient plusieurs outils spécialisés en cybersécurité offensive.

Vérification des outils

Les outils utilisés dans nos scénarios ont été vérifiés :

Vérification de Nmap:

```
(salmafadili@kali)-[~]  
$ nmap --version  
Nmap version 7.98 ( https://nmap.org )  
Platform: x86_64-pc-linux-gnu  
Compiled with: liblua-5.4.8 openssl-3.5.5 libssh2-1.11.1 libz-1.3.1 libpcap-2-  
10.46 libpcap-1.10.6 nmap-libdnet-1.18.0 ipv6  
Compiled without:  
Available nsock engines: epoll poll select
```

Figure 28: Vérification Nmap sur Kali

Vérification de Hydra :

```
(salmafadili@kali)-[~]  
$ hydra -h  
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in  
military or secret service organizations, or for illegal purposes (this is n  
on-binding, these *** ignore laws and ethics anyway).  
  
Syntax: hydra [[[-l LOGIN|-L FILE] [-p PASS|-P FILE]] | [-C FILE]] [-e nsr] [  
-o FILE] [-t TASKS] [-M FILE [-T TASKS]] [-w TIME] [-W TIME] [-f] [-s PORT] [  
-x MIN:MAX:CHARSET] [-c TIME] [-ISOuvVd46] [-m MODULE_OPT] [service://server[  
:PORT]/[OPT]]  
  
Options:  
-R          restore a previous aborted/crashed session  
-I          ignore an existing restore file (don't wait 10 seconds)  
-S          perform an SSL connect  
-s PORT    if the service is on a different default port, define it here  
-l LOGIN or -L FILE login with LOGIN name, or load several logins from FILE  
-p PASS or -P FILE try password PASS, or load several passwords from FILE  
-x MIN:MAX:CHARSET password bruteforce generation, type "-x -h" to get help  
-y          disable use of symbols in bruteforce, see above  
-r          use a non-random shuffling method for option -x  
-e nsr      try "n" null password, "s" login as pass and/or "r" reversed login  
-u          loop around users, not passwords (effective! implied with -x)  
-C FILE     colon separated "login:pass" format, instead of -L/-P options  
-M FILE     list of servers to attack, one entry per line, ':' to specify port  
-D XofY     Divide wordlist into Y segments and use the Xth segment.
```

Figure 29: Vérification Hydra sur Kali

Vérification de Nikto :

```
(salmafadili@kali)-[~]  
$ nikto -Version  
Nikto 2.6.0 (LW 2.5)
```

Figure 30: Vérification Nikto sur Kali

Ces outils permettront de simuler plusieurs cyberattaques réelles contre la machine cible.

3.3 Déploiement de Wazuh SIEM

3.3.1 Présentation de Wazuh

Wazuh est une plateforme open source de détection des menaces, de surveillance de l'intégrité, de réponse aux incidents et de conformité. Elle est constituée de trois composants principaux :

Tableau 2 : Composants de la solution Wazuh

Composant	Rôle	Emplacement dans notre projet
Wazuh Manager	Cœur du SIEM : reçoit, analyse et génère les alertes	PC-3 (Docker)
Wazuh Agent	Collecte les logs et les envoie au Manager	PC-2 (Ubuntu Server)
Wazuh Dashboard	Interface web de visualisation des alertes	PC-3(Docker : port 443)

3.3.2 Installation via Docker Compose

Le déploiement de Wazuh via Docker Compose présente plusieurs avantages : isolation des composants, démarrage en une seule commande, et facilité de mise à jour. La commande de déploiement utilisée est la suivante :

```

14->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp single-node_wazuh.manager_1
hajar@wazuh-manager:~/wazuh-docker/single-node$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
232fa9740a9b   wazuh/wazuh-manager:4.7.2          "/init"                 11 minutes ago Up 44 seconds  0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp single-node_wazuh.manager_1
63d49bb87b01   wazuh/wazuh-indexer:4.7.2          "/entrypoint.sh open..." 11 minutes ago Created                                single-node_wazuh.indexer_1

hajar@wazuh-manager:~/wazuh-docker/single-node$ cd ~/wazuh-docker/single-node
hajar@wazuh-manager:~/wazuh-docker/single-node$ sudo docker-compose down
Stopping single-node_wazuh.manager_1 ... done
Removing single-node_wazuh.manager_1 ... done
Removing single-node_wazuh.indexer_1 ... done
Removing network single-node_default
hajar@wazuh-manager:~/wazuh-docker/single-node$ sudo rm -rf config/wazuh-indexer-ssl-certs
hajar@wazuh-manager:~/wazuh-docker/single-node$ docker-compose -f generate-indexer-certs.yml run --rm generator
Creating network 'single-node_default' with the default driver
Pulling generator (wazuh/wazuh-certs-generator:0.0.1)...
0.0.1: Pulling from wazuh/wazuh-certs-generator
55a86de68c5c: Pull complete
573f4d11a520: Pull complete
edaedc954fb5: Pull complete
8f200922197d: Pull complete
Digest: sha256:ea8b03a68be67bae0e164d82b232eae54dd132c2aacd8d3118ba8705df9364a4
Status: Downloaded newer image for wazuh/wazuh-certs-generator:0.0.1
Creating single-node_generator_run ... done
The tool to create the certificates exists in the in Packages bucket
09/04/2026 09:47:10 INFO: Admin certificates created.
09/04/2026 09:47:10 INFO: Wazuh indexer certificates created.
09/04/2026 09:47:11 INFO: Wazuh server certificates created.
09/04/2026 09:47:11 INFO: Wazuh dashboard certificates created.
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker
hajar@wazuh-manager:~/wazuh-docker/single-node$ sudo chown -R $USER:$USER config/
hajar@wazuh-manager:~/wazuh-docker/single-node$ docker-compose up -d
Creating single-node_wazuh.indexer_1 ... done
Creating single-node_wazuh.manager_1 ... done
Creating single-node_wazuh.dashboard_1 ... done
hajar@wazuh-manager:~/wazuh-docker/single-node$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
d8b0c1f452ed   wazuh/wazuh-dashboard:4.7.2        "/entrypoint.sh"        3 minutes ago Up 2 minutes  443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp single-node_wazuh.dashboard_1
682292f1d1a87   wazuh/wazuh-manager:4.7.2          "/init"                 3 minutes ago Up 3 minutes  0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/tcp single-node_wazuh.manager_1
23ff30f7399a   wazuh/wazuh-indexer:4.7.2          "/entrypoint.sh open..." 3 minutes ago Up 3 minutes  0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp single-node_wazuh.indexer_1
hajar@wazuh-manager:~/wazuh-docker/single-node$ _

```

Figure 31: Résultat commande docker ps

3.3.3 Connexion de l'Agent au Manager

Une fois le Manager déployé, l'agent Wazuh installé sur la machine Ubuntu doit être enregistré et connecté au Manager. Cette connexion s'effectue via le fichier de configuration `/var/ossec/etc/ossec.conf` sur la machine Ubuntu, où l'adresse IP ZeroTier du Manager est renseignée.

3.3.4 Règles de détection utilisées

Wazuh utilise un système de règles numérotées pour catégoriser et attribuer un niveau de criticité (level) aux événements détectés. Les règles pertinentes dans notre projet sont :

Tableau 3 : Règles Wazuh utilisées pour la détection SSH

Rule ID	Description	Level	Déclencheur
5710	SSH : Tentative de connexion échouée	5	Mauvais mot de passe
5712	SSH : Multiples tentatives échouées	8	> 4 tentatives
5763	SSH : Attaque Brute Force détectée	10	> 8 tentatives / 30s
40101	SSH : Brute Force critique	12	Seuil maximal atteint

3.4 Système de notification : Script Python « custom-telegram »

3.4.1 Objectif et motivation

Bien que Wazuh dispose d'un tableau de bord (Dashboard) pour visualiser les alertes, l'équipe de sécurité ne peut pas rester en permanence connectée à l'interface web. Pour répondre à ce besoin de réactivité, nous avons développé un script Python personnalisé permettant de recevoir instantanément les alertes critiques sur un bot Telegram dédié, quel que soit l'endroit où se trouve le membre de l'équipe.

3.4.2 Création du Bot Telegram

La création du bot Telegram s'est effectuée en trois étapes simples :

- Démarrer une conversation avec **@BotFather** sur Telegram
- Utiliser la commande /newbot et suivre les instructions pour nommer le bot
- Récupérer le Token d'authentification et le Chat ID de l'équipe

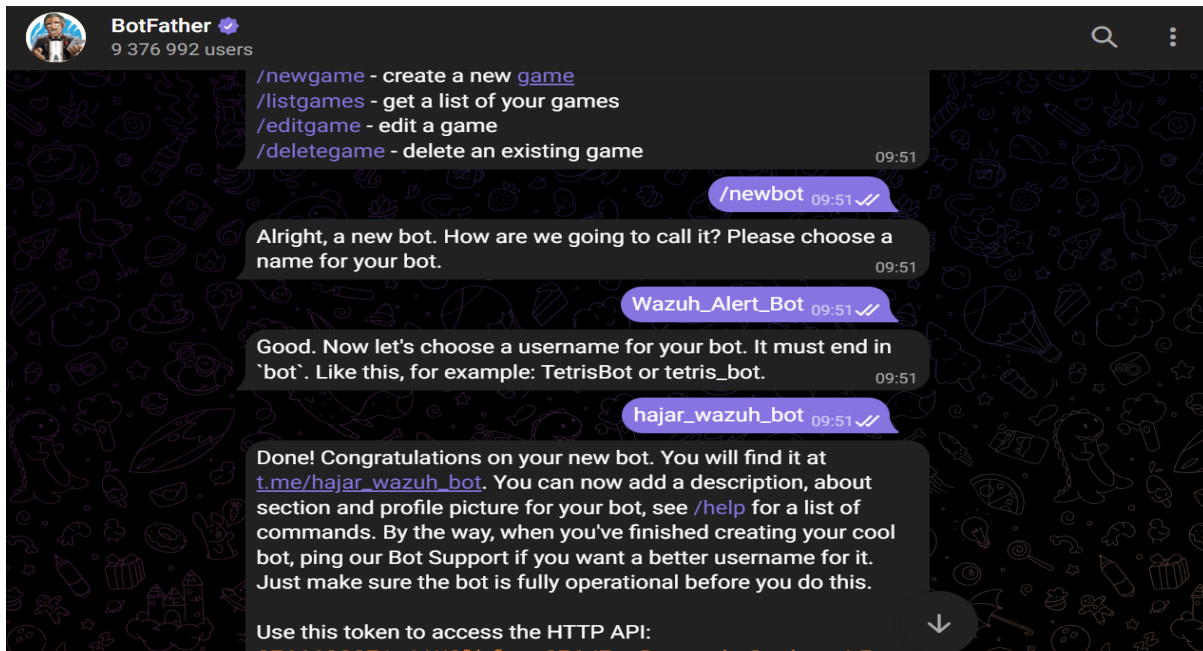


Figure 32: Création du Bot Telegram

3.4.3 Code Python « custom-telegram »

Le script est placé dans le répertoire `/var/ossec/integrations/` sur le serveur Wazuh Manager. Il est invoqué automatiquement par Wazuh dès qu'une alerte atteint le niveau critique configuré.

```
Completed.
root@wazuh:/# ls /var/ossec/integrations/
custom-discord custom-telegram multiverse multiverse.py pagerduty pagerduty.py shuffle shuffle.py slack slack.py viru
root@wazuh:/# cat /var/ossec/integrations/custom-telegram
#!/usr/bin/env python3
import sys, json, urllib.request
with open(sys.argv[1]) as f:
    line = f.readlines()[-1]
    alert = json.loads(line)
TOKEN = '8766690871:AAHG1kfassS26J7r-Bsmr_nkn2_ehgrnLFc'
CHAT_ID = '5935574206'
body = "Wazuh Alert - Level:" + str(alert['rule']['level']) + " - Desc: " + alert['rule']['description']
payload = {'chat_id': CHAT_ID, 'text': body}
data = json.dumps(payload).encode('utf-8')
req = urllib.request.Request("https://api.telegram.org/bot" + TOKEN + "/sendMessage", data=data, headers={'Content-Type': 'appl
urllib.request.urlopen(req)
root@wazuh:/#
```

Figure 33: Script Python custom-telegram

3.4.4 Configuration dans ossec.conf

Pour que Wazuh invoque automatiquement le script lors d'une alerte critique, il est nécessaire d'ajouter le bloc `<integration>` suivant dans le fichier `/var/ossec/etc/ossec.conf` du Manager :

```
GNU nano 7.2 /var/ossec/etc/ossec.conf *
<integration>
  <name>custom-telegram</name>
  <hook_url>https://api.telegram.org/876669871:AAHa2_g15ohEWpbRaVtIhuIr92f20_3x54Y/sendMessage</hook_url>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>
```

Figure 34: Configuration bloc integration ossec.conf

3.4.5 Résultat : Notification reçue sur Telegram

Lorsqu'une alerte de niveau 10 ou supérieur est générée par Wazuh, le script est invoqué automatiquement et un message structuré est immédiatement envoyé sur le bot Telegram de l'équipe. La figure suivante illustre un exemple de notification reçue :



Figure 35: Notification Telegram reçue

3.5 Méthodologie des tests

Afin de valider l'efficacité de notre système de détection et de réponse aux incidents, nous avons adopté une méthodologie structurée basée sur trois scénarios distincts, simulés depuis la machine attaquante Kali Linux vers la machine cible Ubuntu Server, toutes deux interconnectées via le réseau virtuel ZeroTier One.

Chaque scénario est évalué selon trois critères :

- Détection : l'attaque est-elle détectée par Wazuh ?
- Niveau d'alerte : quel level est déclenché ?
- Notification : le message Telegram est-il reçu ?

Tableau des scénarios :

Tableau 4 : Matrice de synthèse des scénarios d'attaques simulés

Scénario	Outil	Cible	Protocol
Reconnaissance & Scan de ports	Nmap	Ubuntu Server	TCP / Tous les ports
Attaque par Force Brute	Hydra	Service OpenSSH	TCP / Port 22
Scan de vulnérabilités Web	Nikto	Serveur web Apache2	TCP / Port 80 (HTTP)

3.6 Scénario 1: Reconnaissance (Nmap)

3.6.1 Présentation de Nmap

Nmap (Network Mapper) est un outil open source de scanning réseau utilisé pour découvrir les hôtes et les services actifs sur un réseau. Il permet d'identifier les ports ouverts, les services en cours d'exécution et le système d'exploitation de la machine cible. Dans le cadre d'une attaque, Nmap constitue la première étape de reconnaissance.

3.6.2 Commande utilisée

La commande suivante a été utilisée : `nmap -A 10.189.31.11`

- -A : active la détection avancée incluant la détection du système d'exploitation, la détection des versions des services, le scan de scripts et le traceroute.

```
(salmafadili@kali)-[~]
$ nmap -A 10.189.31.11
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-19 09:55 +0100
Nmap scan report for 10.189.31.11 (10.189.31.11)
Host is up (0.022s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_ 256 36:8a:a4:03:b1:9d:8b:e0:34:95:68:05:d7:42:bf:9f (ECDSA)
|_ 256 d2:66:ad:dd:85:62:e1:2a:fc:f2:8e:15:d9:65:4e:01 (ED25519)
80/tcp    open  http      Apache httpd 2.4.58 ((Ubuntu))
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: Apache2 Ubuntu Default Page: It works
MAC Address: 5E:0F:96:37:7B:B0 (Unknown)
Device type: general purpose
Running: Linux 5.X
OS CPE: cpe:/o:linux:linux_kernel:5.13
OS details: Linux 5.13
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   21.50 ms  10.189.31.11 (10.189.31.11)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.58 seconds
```

Figure 36: Capture scan Nmap depuis Kali

3.6.3 Détection par Wazuh

Wazuh a détecté le scan réseau et généré des alertes correspondantes sur le Dashboard.

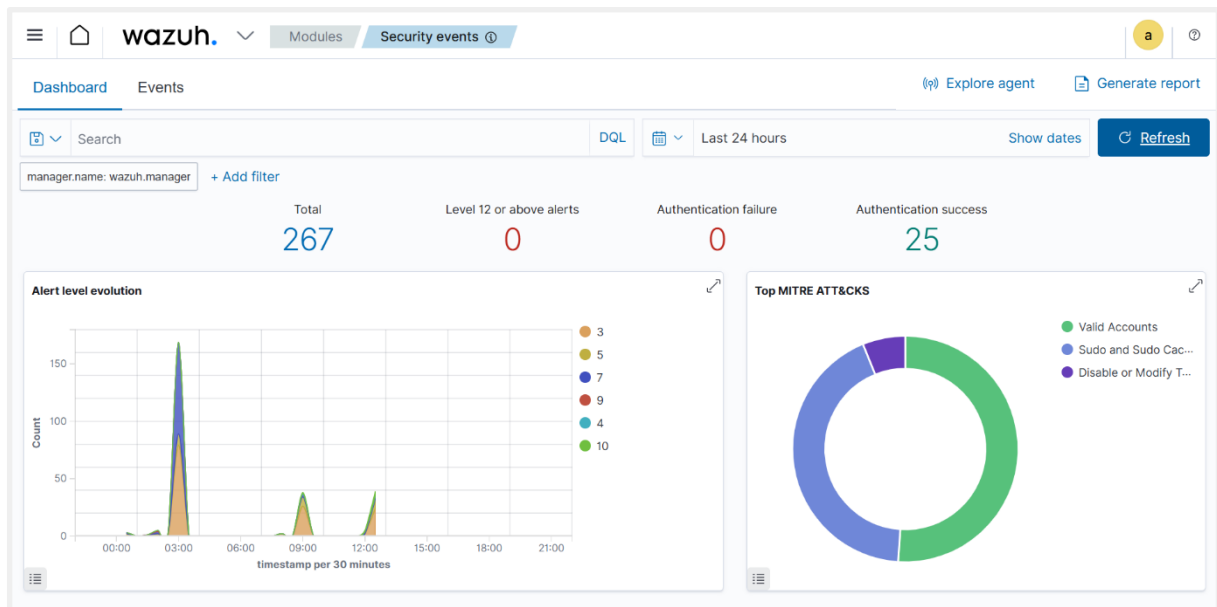


Figure 37: Dashboard Wazuh : Détection Nmap

Security Alerts							
Time ↓	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
> May 18, 2026 @ 12:55:45.544	001	Victime-Ubuntu			Web server 501 error code (Not Implemented).	4	31121
> May 18, 2026 @ 12:55:45.544	001	Victime-Ubuntu			Web server 400 error code.	5	31101
> May 18, 2026 @ 12:55:45.543	001	Victime-Ubuntu			Web server 400 error code.	5	31101
> May 18, 2026 @ 12:55:45.543	001	Victime-Ubuntu			Web server 400 error code.	5	31101
> May 18, 2026 @ 12:55:45.538	001	Victime-Ubuntu			Web server 400 error code.	5	31101
> May 18, 2026 @ 12:55:45.535	001	Victime-Ubuntu			Web server 400 error code.	5	31101
> May 18, 2026 @ 12:55:45.513	001	Victime-Ubuntu			Web server 400 error code.	5	31101
> May 18, 2026 @ 12:55:45.500	001	Victime-Ubuntu			Web server 400 error code.	5	31101

Figure 38: Alertes Wazuh — scan Nmap

3.6.4 Analyse

Ce scénario représente la phase de reconnaissance d'une cyberattaque. Les informations collectées par Nmap (ports ouverts, versions des services) servent de base pour planifier les attaques suivantes.

3.7 Scénario 2 : Brute Force SSH (Hydra)

3.7.1 Présentation de Hydra

Hydra est un outil de test d'intrusion open source spécialisé dans les attaques par force brute sur de nombreux protocoles réseau, dont SSH, FTP, HTTP et bien d'autres. Il teste automatiquement une liste de mots de passe jusqu'à trouver les identifiants valides d'un service cible.

3.7.2 Préparation

Avant de lancer l'attaque, nous avons préparé les éléments suivants :

- **Nom d'utilisateur cible** : testuser
- **Dictionnaire de mots de passe** : rockyou.txt (/usr/share/wordlists/rockyou.txt)
- **Adresse IP cible** : 10.189.31.11
- **Port cible** : 22 (SSH)
- **Threads parallèles** : 4

3.7.3 Commande utilisée

La commande suivante a été utilisée :

```
hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://10.189.31.11 -t 4 -V
```

- l testuser** : nom d'utilisateur cible
- P** : chemin vers le dictionnaire
- t 4** : 4 threads parallèles
- V** : mode verbose, affiche chaque tentative en temps réel
- **ssh** : protocole cible

```

(salmafadili@kali)-[~]
$ hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://10.189.31.11 -t 4 -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or
secret service organizations, or for illegal purposes (this is non-binding, these *** ig
nore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 06:40:35
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) f
rom a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399),
~3586100 tries per task
[DATA] attacking ssh://10.189.31.11:22/
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "123456" - 1 of 14344399 [child 0
] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "12345" - 2 of 14344399 [child 1]
(0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "123456789" - 3 of 14344399 [chil
d 2] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "password" - 4 of 14344399 [child
3] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "iloveyou" - 5 of 14344399 [child
1] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "princess" - 6 of 14344399 [child
0] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "1234567" - 7 of 14344399 [child
3] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "rockyou" - 8 of 14344399 [child
2] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "12345678" - 9 of 14344399 [child
1] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "abc123" - 10 of 14344399 [child
0] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "nicole" - 11 of 14344399 [child
3] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "daniel" - 12 of 14344399 [child
2] (0/0)
[ATTEMPT] target 10.189.31.11 - login "testuser" - pass "babygirl" - 13 of 14344399 [chil
d 1] (0/0)

```

Figure 39: Capture Hydra en cours d'exécution

3.7.4 Détection par Wazuh

Le Wazuh Agent installé sur Ubuntu Server a collecté en temps réel les logs d'échecs d'authentification SSH. Le Wazuh Manager a corrélé ces événements et déclenché les règles suivantes :

- **Rule 5710** : tentative de connexion SSH échouée
- **Rule 5763** : attaque par force brute détectée
- **Level 10** : alerte haute
- **Level 12** : alerte critique après corrélation

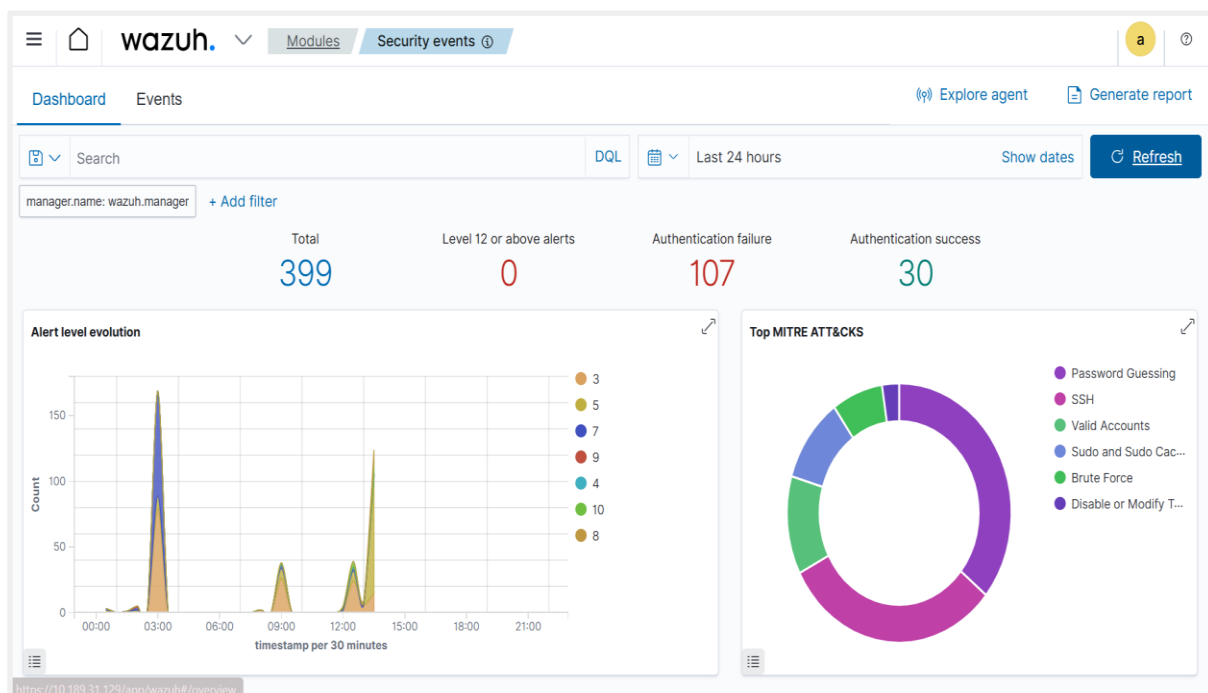


Figure 40: Dashboard Wazuh — Détection Hydra

Security Alerts							
Time ↓	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
> May 18, 2026 @ 13:57:07.890	001	Victime-Ubuntu	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
> May 18, 2026 @ 13:57:07.767	001	Victime-Ubuntu	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
> May 18, 2026 @ 13:57:07.766	001	Victime-Ubuntu	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
> May 18, 2026 @ 13:57:07.265	001	Victime-Ubuntu	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
> May 18, 2026 @ 13:57:07.265	001	Victime-Ubuntu	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710
> May 18, 2026 @ 13:57:07.264	001	Victime-Ubuntu	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: Attempt to login using a non-existent user	5	5710

Figure 41 : :Alertes Wazuh — Brute Force SSH

3.7.5 Notification Telegram

Dès que le niveau d'alerte 10 a été atteint, le script Python custom-telegram a été automatiquement déclenché par Wazuh. Ce script a extrait les informations de l'alerte (Attack Type , Source IP , Country ,Time) et les a envoyées instantanément via l'API Telegram à notre Bot dédié.

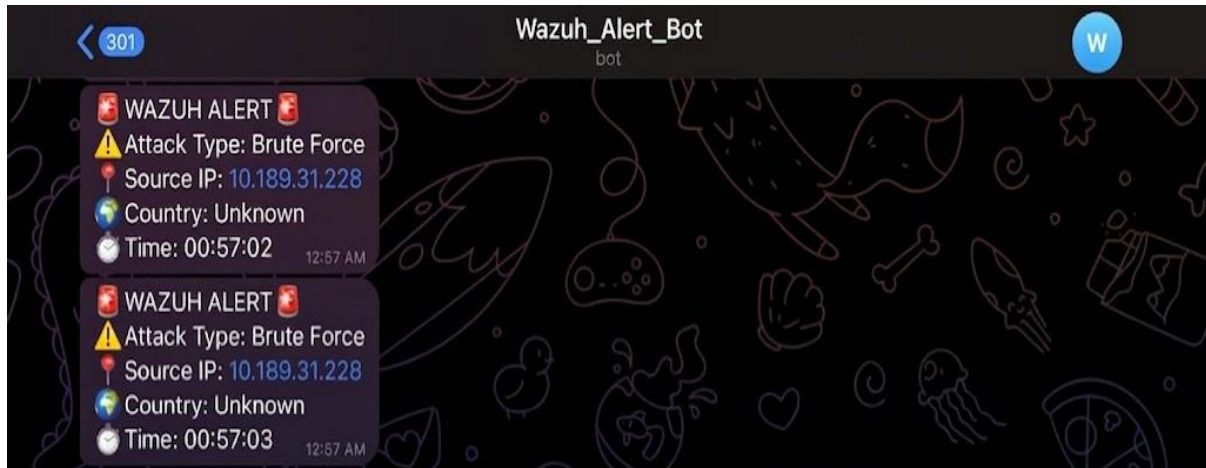


Figure 42: Notification Telegram (Brute Force)

3.7.6 Analyse

Ce scénario valide l'ensemble de la chaîne de sécurité mise en place : de la génération de l'attaque par Hydra, à la détection par Wazuh, jusqu'à la notification instantanée via Telegram. Le temps de détection est quasi immédiat, démontrant l'efficacité du système en conditions réelles

3.8 Scénario 3 : Scan de vulnérabilités (Nikto)

3.8.1 Présentation de Nikto

Nikto est un scanner de vulnérabilités web open source disponible sur Kali Linux. Il analyse les serveurs web à la recherche de failles de sécurité connues, de fichiers dangereux, de versions obsolètes et de configurations incorrectes.

3.8.2 Commande utilisée

La commande suivante a été utilisée : `nikto -h http://10.189.31.11`

-h : spécifie l'hôte cible (serveur Apache)

```
(salmafadili@kali)-[~]
$ nikto -h http://10.189.31.11
- Nikto v2.6.0

+ Target IP: 10.189.31.11
+ Target Hostname: 10.189.31.11
+ Target Port: 80
+ Platform: Linux/Unix
+ Start Time: 2026-05-19 12:23:22 (GMT1)

+ Server: Apache/2.4.58 (Ubuntu)
+ ERROR: Failed to check for updates: 403
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [600050] Apache/2.4.58 appears to be outdated (current is at least 2.4.66).
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 29af, size: 64f8190982a15, mtime: gzip. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [999990] OPTIONS: Allowed HTTP Methods: OPTIONS, HEAD, GET, POST .
```

Figure 43: Capture scan Nikto depuis Kali

3.8.3 Détection par Wazuh

Wazuh a détecté les requêtes anormales générées par Nikto sur le serveur Apache et a généré des alertes correspondantes.

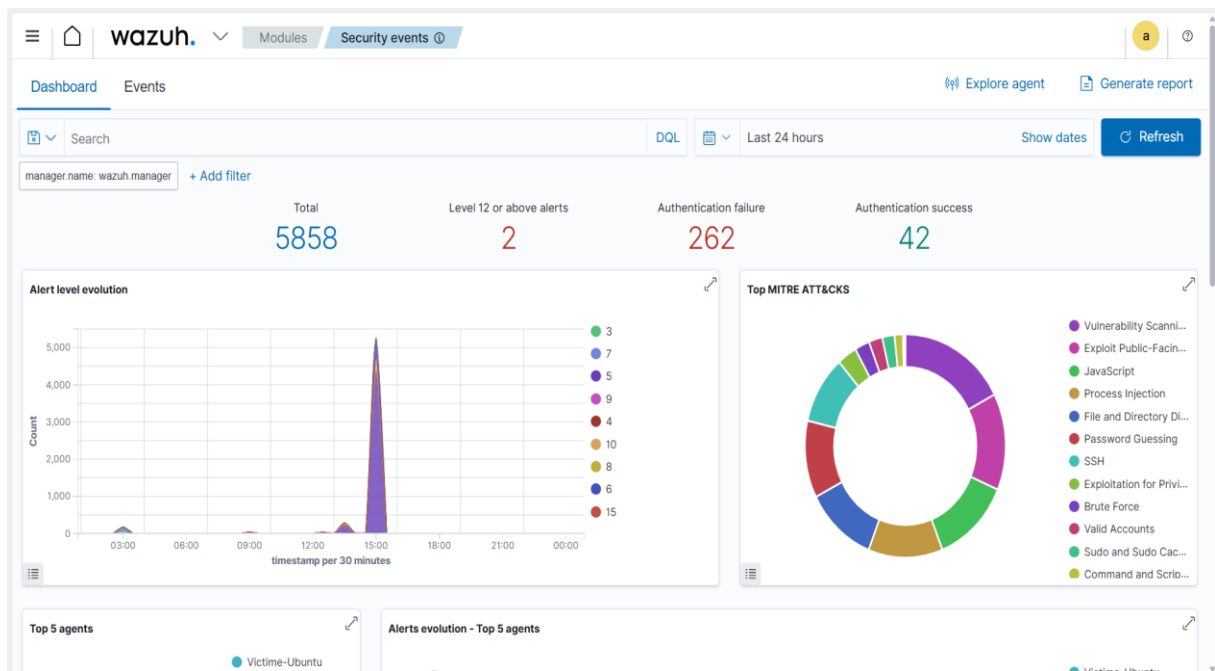


Figure 44: Dashboard Wazuh (Détection Nikto)

Time ↓	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
May 18, 2026 @ 15:25:23.959	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.953	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.948	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.930	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.928	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.912	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.910	001	Victime-Ubuntu	T1595.002	Reconnaissance	Multiple web server 400 error codes from same source ip.	10	31151
May 18, 2026 @ 15:25:23.909	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.908	001	Victime-Ubuntu			Web server 400 error code.	5	31101
May 18, 2026 @ 15:25:23.906	001	Victime-Ubuntu			Web server 400 error code.	5	31101

Figure 45: Alertes Wazuh (scan Nikto)

3.8.4 Notification Telegram

Dès que le niveau d'alerte 10 a été atteint, le script Python custom-telegram a été automatiquement déclenché par Wazuh. Ce script a extrait les informations de l'alerte (Attack Type , Source IP , Country ,Time) et les a envoyées instantanément via l'API Telegram à notre Bot dédié.

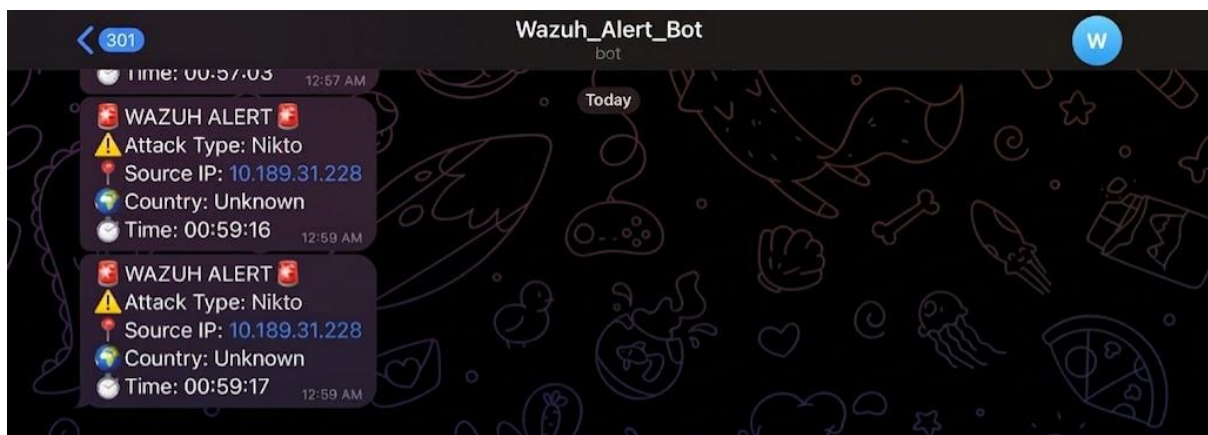


Figure 46: Notification Telegram (Nikto)

3.9 Tableau comparatif des résultats

Le tableau ci-dessus présente les alertes générées par Wazuh lors de nos trois scénarios d'attaques. On remarque que :

- Le scan Nikto a déclenché la Rule 31151 (Level 10 Reconnaissance) lorsque Wazuh a corrélié plusieurs erreurs 400 provenant de la même adresse IP source.
- L'attaque Brute Force SSH via Hydra a déclenché la Rule 5710 (Level 5 → 10/12) classifiée sous la tactique MITRE ATT&CK "Credential Access" avec les techniques T1110.001 et T1021.004.

Tableau 5 : comparatif des résultats

Scénario	Outil	Rule ID	Description	Level	Détectée
Reconnaissance Nmap	Nmap	—	Port scan détecté sur Victime-Ubuntu	5/7	Oui
Scan de vulnérabilités Nikto	Nikto	31101 31151	Web server 400 error code Multiple 400 errors - same IP Tactic:Reconnaissance (T1595.002)	5 / 10	Oui
Brute Force SSH Hydra	Hydra	5710	sshd: login non-existent user Credential Access 1110.001 - T1021.004	5/ 10/12	Oui

3.10 Évaluation de l'efficacité du système

Les tests réalisés ont permis d'évaluer l'efficacité globale de notre système de sécurité selon deux axes principaux.

Points forts identifiés :

- Détection en temps réel des trois types d'attaques (Nmap, Nikto, Hydra) grâce à Wazuh SIEM, avec génération automatique d'alertes classifiées selon le framework MITRE ATT&CK.
- Notification instantanée via Telegram dès qu'une alerte critique est déclenchée, permettant une réponse rapide de l'équipe de sécurité sans nécessiter une surveillance permanente du Dashboard.
- Déploiement moderne et reproductible via Docker et Docker-Compose, reflétant les pratiques actuelles du Cloud-Native.
- Architecture distribuée réaliste grâce à ZeroTier One, simulant un environnement Cloud privé entre trois machines distantes.

Limites identifiées :

- Les règles de détection de Wazuh sont statiques une attaque lente et discrète de type Low & Slow Attack pourrait ne pas être détectée si elle reste sous le seuil des règles configurées.
- Absence de blocage automatique de l'adresse IP attaquante après détection l'intervention reste manuelle.

Dans ce chapitre, nous avons présenté la mise en œuvre pratique de l'environnement Cloud IaaS virtualisé. Nous avons détaillé les différentes étapes de configuration du réseau ZeroTier, de déploiement des machines virtuelles, d'installation de Wazuh SIEM ainsi que la mise en place du système de notification Telegram. Nous avons également présenté la méthodologie de test adoptée afin d'évaluer les capacités de détection et de supervision de l'architecture proposée.

Ce chapitre nous a permis de valider concrètement l'efficacité de notre système de sécurité. Les trois scénarios d'attaques simulés via Nmap, Hydra et Nikto ont tous été détectés avec succès par Wazuh SIEM, avec génération d'alertes en temps réel et envoi de notifications instantanées via Telegram. Ces résultats démontrent la robustesse de l'architecture mise en place et confirment la pertinence des outils choisis pour sécuriser un environnement Cloud IaaS virtualisé.

Conclusion générale

Ce projet de fin d'études nous a permis de concevoir et de mettre en œuvre une solution concrète de détection des cybermenaces dans un environnement Cloud IaaS virtualisé. En combinant des outils open source tels que Wazuh SIEM, Docker, ZeroTier One et PFSense, nous avons réussi à construire une infrastructure distribuée réaliste, sécurisée et opérationnelle.

Les trois scénarios d'attaques simulés reconnaissance via Nmap, scan de vulnérabilités via Nikto, et force brute SSH via Hydra ont tous été détectés avec succès en temps réel par Wazuh, avec des alertes correctement classifiées selon le framework MITRE ATT&CK. Le système de notification automatique via Telegram, développé par notre équipe, a complété cette chaîne de sécurité en garantissant une réactivité immédiate sans surveillance permanente du Dashboard.

Au-delà des résultats techniques, ce projet nous a offert une expérience enrichissante qui a renforcé nos compétences en administration système, en cybersécurité offensive et défensive, et en développement Python appliqué à la sécurité.

Comme perspectives futures, nous envisageons d'enrichir ce système par l'intégration d'un module de Machine Learning pour une détection comportementale plus avancée, ainsi que l'activation de l'Active Response de Wazuh pour un blocage automatique des attaquants.

Bibliographie

- [1] Peter Mell, Timothy Grance, The NIST Definition of Cloud Computing, National Institute of Standards and Technology, Special Publication 800-145, 2011.
- [2] Rajkumar Buyya, James Broberg, Andrzej Goscinski, *Cloud Computing: Principles and Paradigms*, Wiley, 2011.
- [3] Amazon Web Services, *Overview of Amazon Web Services*, AWS Documentation.
- [4] Mastering Cloud Computing, McGraw-Hill, 2013.
- [5] Microsoft, *Microsoft Azure Fundamentals*, Microsoft Learn.
- [6] Cloud Security and Privacy, O'Reilly Media, 2009.
- [7] William Stallings, *Network Security Essentials: Applications and Standards*, Pearson Education, 2017.
- [8] Computer Security: Principles and Practice, Pearson, 2018.
- [9] Network Security Bible, Wiley Publishing, 2011.
- [10] Snort Official Documentation.
- [11] Cisco, *Cisco Secure Firewall and NGIPS Documentation*.
- [12] Hacking Exposed, McGraw-Hill, 2017.
- [13] Web Application Hacker's Handbook, Wiley, 2011.